HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

MASTER THESIS

Energy-Efficient Spiking Neural Networks for Capacitive Image-Based Finger Classification

TRẦN MINH HIỂN
hien.tm241020M@sis.hust.edu.vn

Program: Ambient Computing, Multimedia and Interaction

Supervisor: Dr. Nguyễn Huy Hoàng

School: School of Electrical and Electronic Engineering

HANOI, 05/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

MASTER THESIS

Energy-Efficient Spiking Neural Networks for Capacitive Image-Based Finger Classification

TRẦN MINH HIỂN
hien.tm241020M@sis.hust.edu.vn

Program: Ambient Computing, Multimedia and Interaction

Supervisor: Dr. Nguyễn Huy Hoàng

School: School of Electrical and Electronic Engineering

HANOI, 05/2026

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my family and my loved one for their constant support and encouragement throughout this project. My heartfelt thanks also go to my colleagues at the **Modeling and Simulation Center, Viettel High Tech**, for their valuable insights and inspiration during the research process. I am deeply thankful to my professors, **Assoc.Prof. Nguyễn Đức Minh**, **Dr.Eng. Hoàng Phương Chi**, **Dr.Eng. Nguyễn Huy Hoàng** at the **EDABK Laboratory** for their guidance and academic support. Finally, I would like to acknowledge my own perseverance and dedication in completing this final project. This work represents not only a learning journey but also a personal milestone that I am truly proud of

ABSTRACT

Capacitive sensing technology has become a standard for touch-based interaction, yet most systems treat all finger contacts as uniform inputs. Recogning and classifying specific finger types (e.g., thumb, index, or middle finger) from capacitive images can significantly enhance human-computer interaction and device ergonomics. However, deploying such recognition models on battery-constrained edge devices requires a careful balance between classification accuracy and power efficiency.

This thesis investigates the development of efficient deep learning models for capacitive image-based finger recognition, with a particular focus on Spiking Neural Networks (SNNs)—a bio-inspired and event-driven computing paradigm known for its low power consumption and hardware efficiency. Two SNN-based architectures are developed and evaluated in this study.

First, a RANC-based SNN model is proposed as a fully spiking architecture trained entirely from scratch using SNN principles. The model is designed within the Training, Testing, and Emulation environment provided by RANC, enabling strong potential for deployment on neuromorphic and hardware-accelerated platforms such as FPGAs. The proposed RANC-based SNN demonstrates recognition performance comparable to the CNN baseline introduced in the original CapFingerID study, while significantly reducing computational complexity and model parameters.

Second, a Spiking Vision Transformer (Spiking ViT) architecture is explored to incorporate the powerful representation learning capability of Vision Transformers into the spiking domain. The proposed model adopts a hybrid CNN-SNN design, combining efficient spiking computation with global attention mechanisms to improve feature extraction and classification performance. Experimental results show that the Spiking ViT outperforms the original CapFingerID baseline model by approximately 6% in classification accuracy.

Overall, the findings demonstrate that SNN-based architectures—particularly transformer-enhanced spiking models—provide a promising and energy-efficient alternative to conventional deep learning approaches for finger-type recognition in next-generation interactive and embedded systems.

Student

(Signature and full name)

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Definition
API	Giao diện lập trình ứng dụng (Application Programming Interface)
EUD	Phát triển ứng dụng người dùng cuối(End-User Development)
GWT	Công cụ lập trình Javascript bằng Java của Google (Google Web Toolkit)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
IaaS	Dịch vụ hạ tầng

CHAPTER 1. INTRODUCTION

1.1 Problem Statement

In the current landscape of mobile and wearable technology, capacitive touch sensing serves as the primary medium for human-computer interaction (HCI). While modern interfaces are highly responsive to touch coordinates, they remain largely "finger-unaware,"treating every contact point as a generic input regardless of whether it originates from a thumb, an index, or a middle finger.

The ability to accurately classify specific finger types from raw capacitive images-often referred to as "capacitive heatmapsrepresents a significant leap toward Context-Aware Sensing. Such capability allows devices to dynamically adapt their user interface (UI) based on hand posture, improve ergonomic accessibility for one-handed use, and enable complex gesture shortcuts. However, the realization of this technology faces a critical bottleneck: The Energy-Accuracy Trade-off.

Recognizing finger types requires continuous, "always-on"monitoring of the sensor data. Traditional Deep Learning models, such as Convolutional Neural Networks (CNNs) and the more recent Vision Transformers (ViTs), have set benchmarks in image classification accuracy. Nevertheless, these models are computationally expensive, involving massive floating-point operations that lead to high power consumption. For battery-constrained edge devices like smartwatches or IoT sensors, the energy drain of running such heavy models is often prohibitive.

Consequently, there is an urgent need for a new computing paradigm that can maintain high classification performance while operating within a minimal energy budget. Spiking Neural Networks (SNNs), which emulate the event-driven and sparse communication of the biological brain, offer a promising solution. This research identifies the core problem as the design and optimization of SNN architectures-specifically comparing SNN-equivalents of CNNs and advanced Spiking Vision Transformers-to achieve energy-efficient finger type recognition using the CapfingerId dataset. Solving this problem is essential for bringing sophisticated, context-aware interaction to the next generation of low-power mobile hardware.

1.2 Background and Problems of Research

1.2.1 Current State of Research in Finger Recognition

The field of capacitive finger recognition has transitioned from simple touch-point detection to complex identity and posture classification.

- **Baseline Studies on CapfingerId**: The **CapfingerId** [1] dataset was intro-

duced by **Le and Mayer (2019)**, who demonstrated that Convolutional Neural Networks (CNNs) could achieve over 92% accuracy in differentiating finger types (e.g., thumb vs. index) by treating capacitive raw data as low-resolution images.

- **Sequential and Feature-Based Improvements**: Recent work such as **SpeciFingers (2024)** [2] has expanded on this by utilizing sequential features of touch events to improve identification stability. These studies confirm that spatial heatmaps from capacitive sensors contain enough structural information (size, pressure distribution, and shape) for high-fidelity classification.
- **The Shift to Transformers**: In the broader context of computer vision, **Vision Transformers (ViTs)** have recently outperformed CNNs in capturing global dependencies. According to recent benchmarks in 2025, ViTs exhibit superior robustness in biometric tasks due to their self-attention mechanisms, although they are typically applied to high-resolution RGB data rather than sparse capacitive images.
- **Emergence of Neuromorphic Approaches**: To address the energy constraints of edge devices, researchers have begun exploring **Spiking Neural Networks (SNNs)**. For example, recent developments in **finger vein recognition (2025)** using Adaptive Firing Threshold SNNs (ATSNN) have shown that spike-based encoding can transform static biometric images into energy-efficient spatio-temporal dynamics with minimal parameter counts.

1.2.2 Limitations of Existing Solutions

Despite these advancements, current solutions face three primary bottlenecks that this thesis seeks to address:

- **The Efficiency Gap of Synchronous Models**: Traditional CNNs and ViTs rely on frame-based, synchronous processing. As highlighted in a **2025 MDPI review on SNNs in Imaging** [3], these models execute massive Multiply-Accumulate (MAC) operations regardless of input sparsity. Since capacitive images are inherently sparse (most of the grid is "empty"or "zero"), frame-based models waste significant energy processing non-informative pixels.
- **High Power Consumption for "Always-on"Sensing**: While CNNs achieve high accuracy, their power footprint is often too high for wearable or mobile integration. Research on low-power FPGAs (2024)[4][5] indicates that while traditional deep learning can consume several Watts, neuromorphic models can reduce this to the milliwatt (mW) or even microwatt (μW) range by ex-

ploiting event-driven computation.

- **Structural Complexity vs. Spiking Mechanics**: Integrating the global attention of Transformers into a spiking framework (Spiking ViT) is a cutting-edge challenge. Most existing Spiking ViT designs, such as **SpikeAtConv (2025)** [6], have primarily been tested on large-scale datasets like **ImageNet** [7]. Their applicability and efficiency on small-scale, low-resolution capacitive datasets like *CapfingerId* remain largely unquantified and under-explored.Lack of Comparative Energy Benchmarking: There is a notable absence of empirical studies that directly measure and compare the energy-per-inference ($mJ/inference$) of CNNs vs. SNNs for the specific task of capacitive finger-type classification, leaving a gap in the literature for hardware-aware biometric design.

1.3 Objectives and Conceptual Framework

This study aims to develop energy-efficient deep learning models for capacitive image-based fingerprint recognition, with a particular focus on Spiking Neural Networks (SNNs). Based on the challenges identified in Section 1.2, the thesis defines several key objectives and proposes corresponding methodological directions.

1.3.1 Objectives

The primary objectives of this thesis are as follows:

I. **To design a Spiking Neural Network (SNN) based on the RANC framework**: The model is constructed by following the structural principles and design guidelines suggested in RANC, ensuring consistency in the implementation and evaluation of all SNN-based approaches in this work.

II. **To extend Vision Transformer (ViT) architectures to the spiking domain** : This objective explores the feasibility of integrating transformer-based models into SNN frameworks. A spiking version of Vision Transformer (Spiking ViT) will be proposed to capture long-range dependencies in fingerprint images while maintaining low energy consumption.

1.4 Contributions

This thesis makes the following main contributions:

I. A SNN model based on the RANC framework.

II. A Spiking Vision Transformer.

1.5 Organization of Thesis

The remainder of this thesis is organized as follows.

Chapter 2 presents the problem context and theoretical foundations relevant to this thesis. It begins by defining the research scope, including the capacitive fingerprint classification task, problem formulation, evaluation metrics, and existing research challenges. The chapter then reviews related works on capacitive fingerprint recognition and introduces the CapFingerId dataset, which serves as the primary benchmark throughout this study. In addition, the chapter provides an overview of Spiking Neural Networks (SNNs), including neuron models, spike encoding methods, and their advantages for energy-efficient computing. The RANC framework is subsequently introduced as the hardware-aware neuromorphic architecture considered for the proposed SNN design. Finally, the chapter presents the theoretical background of Vision Transformers (ViTs) and Spiking Vision Transformers (SNN-ViTs), highlighting their potential for learning robust spatial representations from low-resolution capacitive images while maintaining energy-efficient processing.

Chapter 3 presents the proposed methodologies and architectures for capacitive image-based fingerprint recognition. The chapter first introduces a RANC-based Spiking Neural Network (SNN) that employs burst encoding, overlapping segmentation, Tea layers, and spike-based aggregation for efficient spatio-temporal feature extraction. It then presents a Spiking Vision Transformer (Spiking ViT) based on the SpikeFormer framework, incorporating Spiking Patch Splitting (SPS) and Spiking Self-Attention (SSA) to model global dependencies in spike-based representations. In addition, the chapter describes the input encoding process, training strategy, and key architectural design choices for both models.

Chapter 4 presents the experimental evaluation of the proposed RANC-based Spiking Neural Network (SNN) and Spiking Vision Transformer (SNN-ViT) architectures for capacitive fingerprint recognition. The experiments analyze the effects of spatial segmentation, Tea layer output dimensionality, transformer depth, embedding dimension, and temporal encoding windows on recognition performance and model complexity. The results show that overlapping segmentation and temporal spike encoding improve the effectiveness of the RANC-based SNN, while deeper transformer structures and larger embedding dimensions enhance the global feature modeling capability of the SNN-ViT. Overall, the experimental findings demonstrate the effectiveness of combining spiking computation, temporal encoding, and transformer-based attention mechanisms for efficient and accurate capacitive fingerprint recognition.

Chapter 5 concludes the thesis by summarizing the main findings and discussing the achieved results. It also provides insights into the limitations of the current

work and proposes directions for future research. In particular, future work will emphasize the deployment of Spiking Neural Network (SNN) models on hardware platforms to fully exploit their potential for energy-efficient computation.

CHAPTER 2. LITERATURE REVIEW

Touchscreen interaction is widely used in modern mobile devices, yet it is typically limited to two-dimensional touch inputs, restricting richer interaction capabilities. Recent studies have explored the use of capacitive images to identify which finger interacts with the screen, with the CapFingerId dataset providing a large-scale benchmark for this task. While convolutional neural networks (CNNs) achieve high classification accuracy, they often require significant computational resources. This thesis therefore investigates energy-efficient alternatives using Spiking Neural Networks (SNNs), including a model developed on the RANC[8] framework with potential for FPGA deployment, although hardware implementation is beyond the scope of this work. In addition, a SpikeFormer-based SNN Vision Transformer is explored, leveraging direct training and strong representation learning capabilities instead of ANN-to-SNN conversion. This chapter presents the problem context, reviews related works, and introduces the necessary theoretical background.

2.1 Scope of Research

2.1.1 Task Description

This thesis addresses the problem of classifying capacitive fingerprint images collected from touchscreen devices. Unlike traditional input modalities that only capture 2D touch locations, capacitive sensing enables the acquisition of low-resolution fingerprint-like images, which contain distinctive patterns related to different fingers. The CapFingerId [1] dataset provides a large-scale collection of such images, comprising various finger combinations, particularly focusing on commonly used fingers such as the thumb and index finger.

However, capacitive images present several challenges compared to conventional optical fingerprint images. Specifically, they are characterized by low resolution, high noise levels, and significant variations in touch position and pressure. Furthermore, the task requires the model to learn position-invariant features in order to robustly distinguish between different fingers under varying interaction conditions. These characteristics make the classification problem non-trivial and motivate the need for robust and efficient learning approaches.

In this thesis, both the proposed RANC-based Spiking Neural Network (SNN) and the Spiking Vision Transformer (Spiking ViT) are designed to perform multi-class finger classification on capacitive touch images. The classification tasks include distinguishing between:

I. Left and right thumbs,

II. Left and right index fingers,

III. Thumb and index finger categories,

IV. Thumb and non-thumb finger categories,

V. Different fingers belonging to the same hand.

These classification settings enable comprehensive evaluation of the proposed architectures under multiple levels of finger discrimination difficulty, ranging from coarse-grained categorization to fine-grained finger identification.

Although the CapFingerId dataset was collected under multiple interaction activities such as tapping, swiping, and other touchscreen gestures, this research focuses exclusively on samples generated from *tap* interactions. The tapping scenario provides relatively stable finger contact patterns and allows more controlled evaluation of the proposed models without the additional motion variability introduced by dynamic gestures.

2.1.2 Problem Formulation

Formally, let the dataset be defined as:

$$D = \{(x_i, y_i)\}_{i=1}^{N} \tag{2.1}$$

where x_i denotes the input capacitive image and $y_i \in \{1, \ldots, K\}$ represents the corresponding class label. The objective is to learn a mapping function:

$$f(x; \theta) \rightarrow y \tag{2.2}$$

where θ denotes the model parameters. The model is trained to minimize the cross-entropy loss for classification.

2.1.3 Evaluation Metrics

The performance of the proposed models is evaluated primarily using classification accuracy. Accuracy measures the proportion of correctly classified samples over the total number of evaluated samples and is defined as:

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N_{\text{total}}}$$

where N_{correct} denotes the number of correctly predicted samples and N_{total} represents the total number of testing samples.

Accuracy is selected as the primary evaluation metric because the classification tasks considered in this thesis are formulated as multi-class finger classification problems with balanced evaluation settings. The metric provides a straightforward and interpretable measure of the overall recognition capability of the proposed architectures.

In addition to classification accuracy, the number of trainable parameters is also reported to evaluate the computational complexity of different model configurations. Since the proposed architectures are intended for lightweight and resource-constrained neuromorphic systems, parameter efficiency is considered an important factor alongside recognition performance.

Therefore, the experimental analysis focuses on the trade-off between classification accuracy and model complexity under different architectural and temporal encoding configurations.

2.1.4 Constraints and Research Gap

While convolutional neural networks (CNNs) have demonstrated strong performance on similar tasks, they often require significant computational resources. In contrast, Spiking Neural Networks (SNNs) offer a more energy-efficient alternative but remain underexplored for capacitive fingerprint classification. Furthermore, the application of transformer-based models within the SNN paradigm is still limited. These challenges motivate the development of efficient and accurate models in this thesis.

2.2 Related Works

2.2.1 Fingerprint Recognition using Capacitive Data

a, Motivation

Early approaches to finger identification often relied on external sensors or additional hardware, which introduces practical limitations in terms of cost, usability, and deployment [1]. In contrast, capacitive touchscreen technology inherently captures variations in electrical capacitance caused by the ridge and valley structures of human fingerprints, allowing it to generate fingerprint-like patterns without requiring dedicated biometric sensors [9], [10]. This capability has motivated recent research to utilize capacitive images as a lightweight and readily available source of biometric information for finger classification and user interaction.

Furthermore, prior work has demonstrated that capacitive images can be effectively leveraged using deep learning techniques. For instance, the CapFingerId study shows that convolutional neural networks can achieve over 92% accuracy in

distinguishing between different fingers using only low-resolution capacitive images [1]. Similarly, more recent work such as SpeciFingers highlights the potential of learning spatio-temporal features from capacitive data to improve finger identification performance [2]. These results indicate that capacitive fingerprint recognition is both feasible and promising for enhancing human-computer interaction.

Despite these advances, capacitive fingerprint data remains challenging due to its low resolution, high noise levels, and sensitivity to touch variations such as position and pressure. These challenges motivate the exploration of more robust and efficient models, particularly energy-efficient approaches such as Spiking Neural Networks, which are better suited for deployment on edge devices.

b, CapFingerId Dataset

To support the training and evaluation process, the CapFingerId [1] dataset was constructed and publicly released to the research community.

The data were collected from 20 participants (15 male and 5 female) with an average age of 22.4. The experimental device was an **LG Nexus 5** smartphone, modified at the kernel level to access raw data from the **Synaptics ClearPad 3350** touch sensor. Participants performed three fundamental touch interactions using all ten fingers: tapping, dragging, and scrolling in various directions.

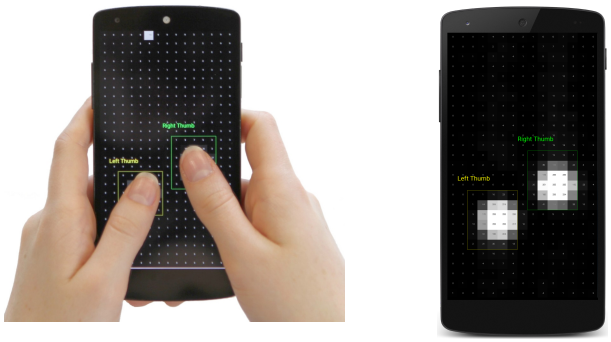
In total, 921,538 raw capacitive images were recorded at a rate of 20 frames per second. After preprocessing, including filtering invalid samples and applying blob detection to identify touch regions, the final dataset consists of 455,709 "blob images."

Each capacitive image has a resolution of 15×27 pixels, where each pixel corresponds to an area of 4.1 mm $\times$ 4.1 mm on the touchscreen surface. Pixel values represent changes in capacitance (measured in pF) relative to a baseline.

The dataset is labeled for all ten fingers, enabling various classification scenarios. Common tasks include distinguishing between left and right thumbs, thumb versus index finger, and full classification across all ten fingers.

2.2.2 Spiking Neural Networks (SNN)

Spiking Neural Networks (SNNs) represent the third generation of neural network models, inspired by the dynamics of biological neurons. Unlike conventional artificial neural networks (ANNs) that process continuous values, SNNs operate using discrete spike events over time. This event-driven mechanism allows computations to occur only when spikes are present, leading to significantly reduced energy consumption. As a result, SNNs have emerged as a promising approach for



(a) Using two thumbs for interaction

(b) Raw Data

Figure 2.1: Identifying left and right thumbs on a commoditysmartphone using the raw capacitive data of touches.

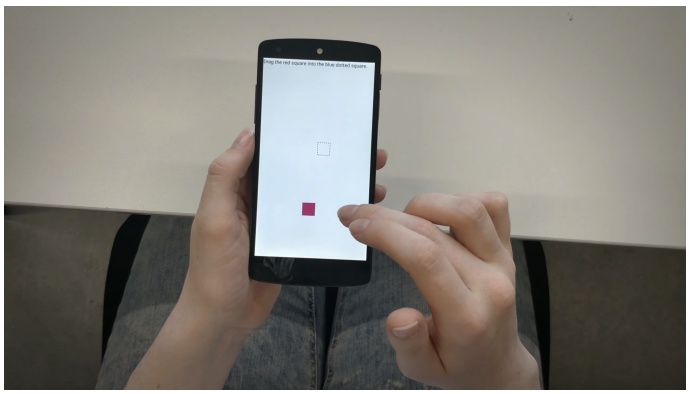


Figure 2.2: Volunteers carried out predefined tasks for data collection purposes

energy-efficient computing, particularly in edge devices and neuromorphic hardware systems.

a, Neuron Model

A fundamental component of SNNs is the spiking neuron model. In this work, the Leaky Integrate-and-Fire (LIF) neuron is adopted due to its simplicity and effectiveness. The membrane potential of a neuron accumulates incoming spikes over time and decays gradually. When the membrane potential exceeds a predefined threshold, the neuron emits a spike and resets its state. This behavior can be described as:

$$\tau \frac{dV(t)}{dt} = -V(t) + I(t) \tag{2.3}$$

where $V(t)$ is the membrane potential, $I(t)$ is the input current, and τ is the membrane time constant.

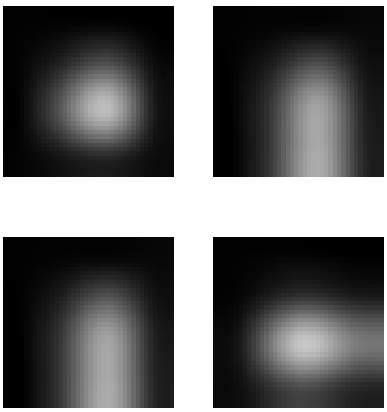


Figure 2.3: Representative samples from dataset.

b, Spike Encoding

To process static image data within the SNN framework, it is necessary to convert input images into spike trains. In this work, *burst coding* is adopted as the primary encoding scheme. Unlike traditional rate coding, burst coding represents input intensity using short bursts of spikes, where both the number of spikes within a burst and their temporal spacing carry information. Stronger input signals typically generate bursts with higher spike counts and shorter inter-spike intervals. This encoding strategy enables richer temporal dynamics while maintaining relatively low latency.

Burst coding offers several advantages over conventional rate coding. By transmitting multiple spikes in rapid succession, it can improve robustness to noise and enhance signal reliability, especially in early layers of the network [11]. Moreover, burst-based representations can achieve a better trade-off between energy efficiency and information fidelity, as meaningful information is conveyed in compact temporal patterns rather than long spike trains [12].

In addition to burst coding, several alternative encoding schemes have been proposed to improve efficiency and temporal representation. *Rate coding* encodes pixel intensity as the firing frequency over a fixed time window and is widely used due to its simplicity and robustness [13], [14]. However, it often requires longer simulation time to achieve high accuracy.

Temporal coding encodes information in the precise timing of spikes rather than their frequency. For example, in *time-to-first-spike* coding, stronger input signals generate earlier spikes, allowing for faster inference with fewer spikes [15], [16]. This approach is more biologically plausible and can significantly reduce latency.

Another related approach is *latency coding*, where input intensity is represented

by the delay before a neuron emits its first spike. While effective for low-latency systems, it may be sensitive to input noise [17].

Population coding represents a single input value using a group of neurons with overlapping receptive fields, enabling a distributed and potentially more robust representation [18]. However, this comes at the cost of increased neuron count and computational complexity.

Finally, *event-based encoding* is commonly used in neuromorphic vision sensors, where spikes are generated only when changes in the input signal occur. Although highly efficient, it is less directly applicable to static image datasets such as CapFingerId [19].

Overall, burst coding is selected in this work due to its ability to capture both rate and temporal information, providing a balanced trade-off between accuracy, latency, and energy efficiency for static image-based SNN applications.

c, Comparison of Spiking Neuron Models

Various spiking neuron models have been proposed in the literature, each offering different trade-offs between biological realism, computational efficiency, and suitability for deep learning.

The **Integrate-and-Fire (IF) model** is the simplest formulation, in which the membrane potential accumulates incoming inputs without any leakage mechanism. Its main advantage lies in its computational efficiency and ease of implementation. However, the absence of a decay term makes it less biologically plausible and can lead to unbounded accumulation of membrane potential, which may negatively affect network stability [13].

The Leaky **Integrate-and-Fire (LIF) model** extends IF by incorporating a leakage term that gradually decays the membrane potential over time. This simple yet effective modification significantly improves temporal dynamics while maintaining low computational cost. As a result, LIF achieves a favorable balance between biological plausibility and efficiency, making it one of the most widely adopted neuron models in modern SNN research [13], [20]. Furthermore, LIF neurons integrate naturally with surrogate gradient methods, enabling stable end-to-end training in deep spiking networks [20].

More advanced variants such as the **Parametric LIF (PLIF)** introduce learnable parameters, such as the membrane time constant, allowing the neuron to adapt its temporal dynamics during training. This flexibility can improve performance across different tasks and network depths, but comes at the cost of increased model

complexity and additional training overhead [21].

Similarly, the **Adaptive LIF (ALIF) model** incorporates dynamic threshold modulation to simulate neuronal adaptation. This enables better modeling of long-term temporal dependencies, but also increases computational complexity and may complicate optimization [22].

At the other end of the spectrum, biologically detailed models such as the **Izhikevich and Hodgkin–Huxley neurons** provide highly accurate representations of neuronal behavior. These models are capable of reproducing a wide range of spiking patterns observed in biological systems. However, their high computational cost and complex dynamics make them impractical for large-scale deep learning applications, particularly in vision-based tasks [23], [24].

Considering the trade-offs discussed above, the **Leaky Integrate-and-Fire (LIF) neuron** is adopted in this work due to its balance between computational efficiency and temporal modeling capability. Its simple formulation enables efficient implementation and stable training using surrogate gradient methods, which is crucial for large-scale deep SNN architectures. While more advanced neuron models offer greater expressiveness, they often introduce unnecessary complexity for the scope of this study. Therefore, LIF is selected to ensure scalability and efficiency, with further exploration of more sophisticated neuron dynamics left for future work.

d, Advantages and Challenges

SNNs offer several advantages over traditional neural networks. Their event-driven nature enables sparse computation, resulting in lower power consumption and improved efficiency, especially when deployed on neuromorphic hardware. However, SNNs also present several challenges, including difficulties in training, sensitivity to encoding schemes, and potential performance gaps compared to conventional CNNs.

Despite these challenges, the potential of SNNs for energy-efficient learning makes them a compelling choice for this work. In particular, their compatibility with neuromorphic frameworks motivates the exploration of hardware-aware implementations, as discussed in the following section.

2.2.3 Neuromorphic Hardware & Framework (RANC)

a, Overview of RANC[8] Architecture

The **Reconfigurable Architecture for Neuromorphic Computing (RANC)** is a highly scalable and flexible hardware ecosystem designed to optimize the execution of Spiking Neural Networks (SNNs). Unlike traditional von Neumann archi-

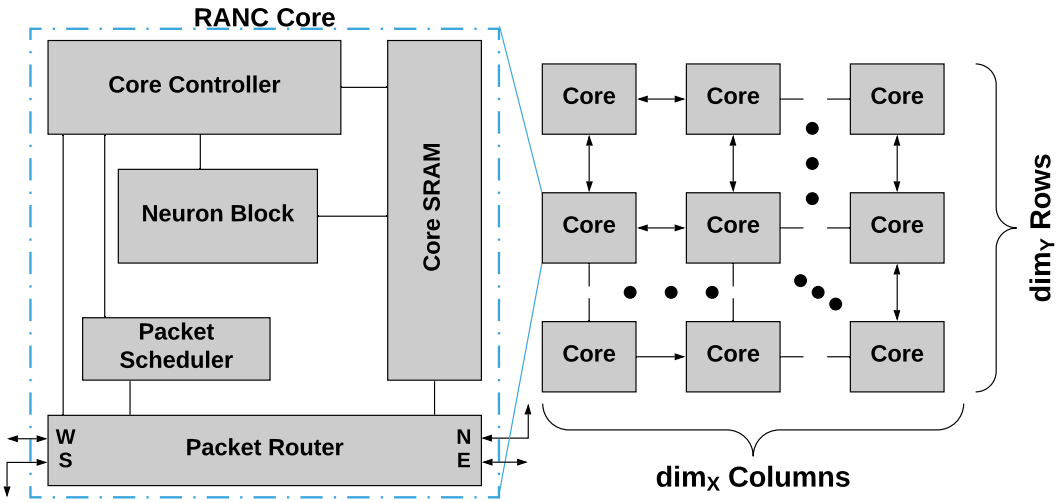


Figure 2.4: High level architectural overview of RANC[8] components.

tectures, RANC[8] adopts a distributed and event-driven design paradigm, where computations are performed locally within processing units and communication occurs only when spike events are generated.

From a structural perspective, the RANC[8] system is composed of multiple interconnected processing elements, referred to as neuron cores. Each core integrates a group of spiking neurons along with their associated synaptic weights and local SRAM memory for storing configuration parameters. These cores operate in parallel and are synchronized through a global clock signal (global tick), enabling efficient large-scale neural computation.

Communication between neuron cores is implemented through an on-chip network (NoC) based on a 2D mesh topology Figure 2.4, utilizing a packet-based routing mechanism. In this scheme, when a neuron generates a spike, the destination information is retrieved from the core's SRAM to construct a packet. Each packet contains four key fields: signed relative offsets dx and dy used for routing the packet to the target core, an axon index to identify the receiving neuron, and a delivery tick offset that specifies the time at which the spike will be processed by the scheduler. The event-driven communication mechanism, combined with deterministic XY routing, minimizes unnecessary data movement and significantly improves the overall energy efficiency of the system.

A key advantage of RANC[8] lies in its strong reconfigurability. The architecture allows flexible mapping of various neural network topologies and configurations onto the hardware through adjustable parameters such as the number of neurons, the number of axons per core, and the precision of synaptic weights. This feature clearly distinguishes RANC[8] from fixed-function neuromorphic systems

implemented as ASIC chips, making it an ideal platform for research, prototyping, and optimization of novel SNN models before deployment on silicon.

Overall, the combination of distributed core architecture, event-driven communication, and high reconfigurability enables RANC[8] to serve as an effective platform for implementing energy-efficient spiking neural networks, particularly in edge computing environments.

b, Neuron and Synapse Mapping

In the RANC[8] ecosystem, the mapping of neurons and synapses is implemented based on a flexible crossbar-based architecture. Each Neuron Block emulates a crossbar connecting $N(a)$ input axons to $N(n)$ output neurons, allowing for a maximum of $N(a)\times N(n)$ synaptic connections per core. All configuration parameters—including weights, activation thresholds, and voltage potentials—are stored centrally within the local core SRAM. The primary advantage of RANC[8] over fixed-function architectures, such as IBM's TrueNorth, lies in its ability to customize the core structure to optimize mapping efficiency for specific applications.

- **Vector-Matrix Multiplication (VMM)**: By supporting symmetric threshold behavior, RANC[8] enables the mapping of signed calculations without the need for complex and resource-heavy feedback systems.This optimization reduces the required neuron and axon footprint by approximately **55-57%**.
- **Convolutional Neural Networks (CNNs)**: RANC[8] allows for the deployment of heterogeneous, large-scale cores (e.g., 1024 axons × 256 neurons). This mechanism significantly improves neuron utilization—increasing it from 0.7% to 94.5%—by minimizing the need for extensive input data repetition across multiple cores.

Through the optimization of core sizes to fully utilize BRAM primitives on the Alveo U250 FPGA, the architecture can scale to emulate as many as 259,072 distinct neurons and over 73.3 million distinct synapses on a single hardware device.

c, Reconfigurability and Flexibility

A key strength of the RANC[8] architecture lies in its high degree of reconfigurability and flexibility, which enables efficient support for a wide range of Spiking Neural Network (SNN) models. Unlike fixed-function neuromorphic systems, RANC[8] allows users to adapt both the network structure and hardware configuration according to specific application requirements.

As illustrated in Fig. 2.5, the RANC[8] ecosystem provides a unified workflow

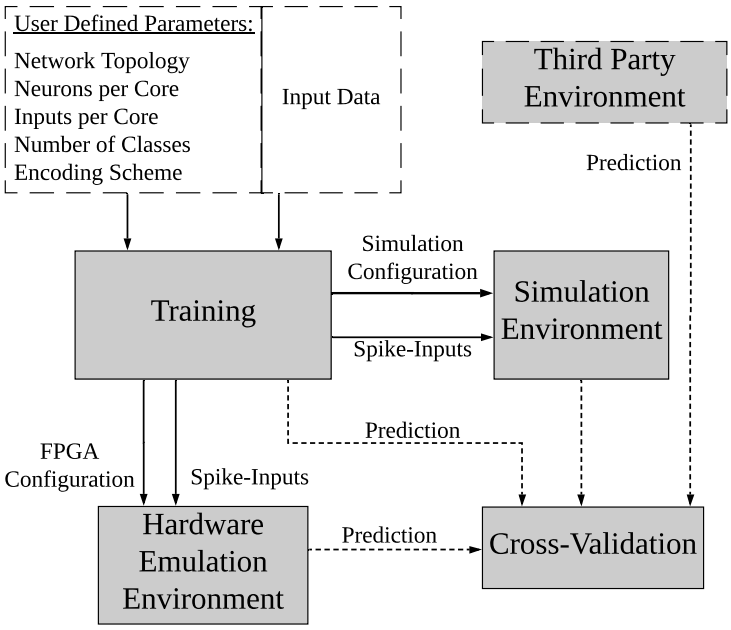


Figure 2.5: Training, Testing, and Emulation environment of offered by RANC.

that integrates training, simulation, and hardware emulation. Users can define multiple parameters, including network topology, number of neurons per core, number of inputs (axons), number of output classes, and the spike encoding scheme. These parameters directly influence how the SNN model is mapped onto the underlying hardware architecture, enabling flexible exploration of different design configurations.

At the architectural level, this flexibility is achieved through configurable neuron cores and programmable routing mechanisms. Each neuron core can be customized in terms of its internal structure and resource allocation, allowing the system to accommodate networks of varying sizes and complexities. In addition, the packet-based communication model enables dynamic routing of spike events between cores, removing the need for fixed interconnections and supporting diverse connectivity patterns.

Furthermore, the RANC[8] framework supports multiple execution environments, as shown in Fig. 2.5, including a simulation environment, hardware emulation, and deployment on FPGA. This multi-stage pipeline allows models to be trained and validated in software before being translated into hardware configurations. The separation between training and deployment ensures that different system configurations can be evaluated efficiently without modifying the core hardware design.

Another important aspect of flexibility is the ability to reuse the same hardware platform for different SNN models. By simply modifying configuration parame-

ters and input encoding schemes, researchers can experiment with various architectures without redesigning the system. This makes RANC[8] particularly suitable for rapid prototyping and iterative development.

Overall, the reconfigurability and flexibility of RANC[8] not only improve hardware utilization but also provide a practical framework for bridging the gap between SNN model development and hardware deployment. This characteristic is highly relevant to this thesis, where different SNN architectures are explored under a unified, hardware-aware design perspective.

d, FPGA Implementation

Field-Programmable Gate Arrays (FPGAs) provide a practical and flexible platform for implementing neuromorphic architectures such as RANC. Unlike application-specific integrated circuits (ASICs), FPGAs allow rapid prototyping and iterative design, making them particularly suitable for research and development of Spiking Neural Networks (SNNs). The reconfigurable nature of FPGAs aligns well with the design philosophy of RANC, enabling efficient mapping of neuron cores, synaptic connections, and routing networks onto hardware.

In the context of RANC, the architecture can be synthesized and deployed on FPGA devices by implementing neuron cores as parallel processing modules and the on-chip communication network as configurable routing logic. The use of packet-based dimension-order (XY) routing simplifies the hardware implementation, as spike events can be efficiently encoded, transmitted, and scheduled using digital logic. Furthermore, the global tick mechanism ensures synchronized operation across all processing elements, maintaining timing accuracy throughout the network. One of the key advantages of FPGA-based implementation is the ability to evaluate energy efficiency and performance trade-offs under realistic hardware constraints. By exploiting parallelism and event-driven computation, SNNs deployed on FPGA platforms can achieve significant reductions in power consumption compared to conventional GPU-based implementations. This makes FPGA-based neuromorphic systems particularly attractive for edge computing applications, where energy and latency are critical considerations. However, FPGA implementations also introduce certain challenges. Resource limitations, such as the availability of Block RAM (BRAM) and Look-Up Tables (LUTs), can restrict the scale of deployable SNN models. Additionally, efficient mapping from high-level SNN models to hardware requires careful design to balance accuracy, latency, and hardware utilization. In this thesis, although the proposed SNN model is developed with consideration for the RANC[8] architecture, the actual deployment on

FPGA hardware is beyond the scope of this work. Nevertheless, the compatibility of the proposed model with RANC[8] highlights its potential for future implementation on reconfigurable hardware platforms, paving the way for energy-efficient and deployable fingerprint recognition systems.

In this thesis, although the proposed SNN model is developed with consideration for the RANC[8] architecture, the actual deployment on FPGA hardware is beyond the scope of this work. Nevertheless, the compatibility of the proposed model with RANC[8] highlights its potential for future implementation on reconfigurable hardware platforms, paving the way for energy-efficient and deployable fingerprint recognition systems.

2.2.4 Vision Transformer & SNN-ViT

a, Vision Transformer

Vision Transformers (ViTs) have recently emerged as a powerful alternative to convolutional neural networks (CNNs) for image recognition by leveraging self-attention mechanisms to model global dependencies across an entire image. Unlike CNNs, which rely on local receptive fields and strong inductive biases such as translation invariance, ViTs enable direct modeling of long-range relationships between image regions, making them particularly effective for capturing distributed and non-local patterns. This characteristic is especially relevant for challenging data such as low-resolution and noisy capacitive images, where discriminative features may not be confined to local neighborhoods. However, ViTs typically require large-scale training data and incur high computational cost, which limits their applicability in resource-constrained environments. These limitations motivate further exploration of efficient variants, including the integration of ViTs with energy-efficient paradigms such as Spiking Neural Networks (SNNs) [25], [26].

The core idea of a Vision Transformer (ViT) is to treat an image as a sequence of patches, analogous to tokens in natural language processing. Given an input image $x \in \mathbb{R}^{H \times W \times C}$, the image is divided into a grid of non-overlapping patches, each of size $P \times P$. The total number of patches is $N = \frac{HW}{P^2}$. Each patch is then flattened into a vector and projected into a latent embedding space through a learnable linear transformation.

To retain spatial information, positional embeddings are added to the patch embeddings. In addition, a learnable classification token, denoted as x_{cls}, is prepended to the sequence. The final input sequence to the Transformer encoder can be written as:

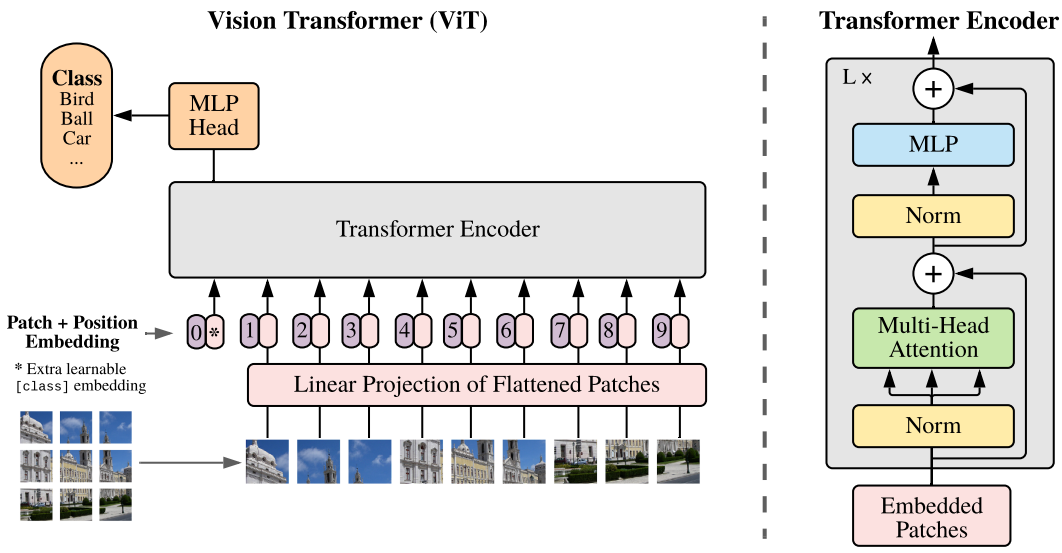


Figure 2.6: Vision Transformer Overview.

$$Z_0 = [x_{\text{cls}}; x_1 E; x_2 E; \ldots; x_N E] + E_{\text{pos}}, \tag{2.4}$$

where x_i denotes the i-th image patch, E is the patch embedding projection matrix, and E_{pos} represents the positional embeddings. The resulting sequence of embeddings serves as the input to a standard Transformer encoder.

The Transformer encoder consists of a stack of identical layers, each composed of a multi-head self-attention (MHSA) mechanism and a position-wise feed-forward network (FFN), combined with residual connections and layer normalization. The self-attention mechanism allows each patch to attend to all other patches in the image, enabling the model to learn global contextual relationships. Specifically, attention weights are computed based on the similarity between query, key, and value representations derived from the input embeddings, allowing the model to dynamically focus on relevant regions of the image. The use of multiple attention heads further enhances the model's capacity to capture diverse patterns and interactions across different representation subspaces.

After passing through the Transformer encoder, the representation corresponding to the classification token is used as a global image descriptor. This representation is then fed into a multi-layer perceptron (MLP) head to produce the final class prediction. Compared to CNN-based approaches, which progressively aggregate local features through convolution and pooling operations, ViTs perform global reasoning at every layer, providing a more flexible and expressive modeling framework.

Despite these advantages, Vision Transformers also present several limitations.

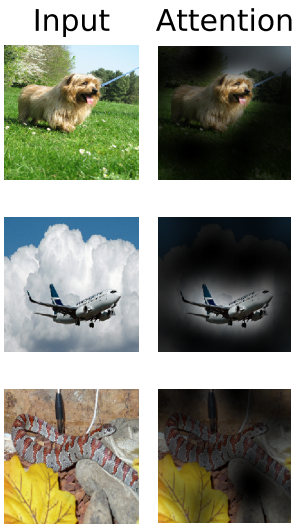


Figure 2.7: Representative examples of attention from the output token to the input space.

In particular, the lack of strong inductive biases makes them less data-efficient than CNNs, often requiring large-scale datasets to achieve competitive performance. Additionally, the quadratic complexity of the self-attention mechanism with respect to the number of patches leads to increased computational and memory costs, which can be prohibitive for deployment on edge devices. These challenges are especially relevant in the context of this thesis, where energy efficiency and hardware constraints are key considerations.

In the context of capacitive fingerprint recognition, the ability of ViTs to model global relationships is particularly beneficial. Capacitive images are characterized by low resolution, high noise levels, and significant variability in touch position and pressure, making it difficult for purely local feature extractors to capture discriminative patterns. By leveraging self-attention, ViTs can integrate information across the entire spatial domain, potentially improving robustness and classification accuracy. Nevertheless, the high energy consumption of conventional ViTs motivates the exploration of more efficient architectures. This leads to the investigation of Spiking Neural Network-based Vision Transformers (SNN-ViT), which aim to combine the representational power of Transformers with the energy efficiency of event-driven computation.

b, SNN-ViT

Recent advances in neuromorphic computing have motivated the integration of Transformer architectures into the spiking domain, leading to the emergence of Spiking Vision Transformers (SNN-ViTs). These models aim to combine the global representation capability of Vision Transformers (ViTs) with the energy efficiency of Spiking Neural Networks (SNNs).

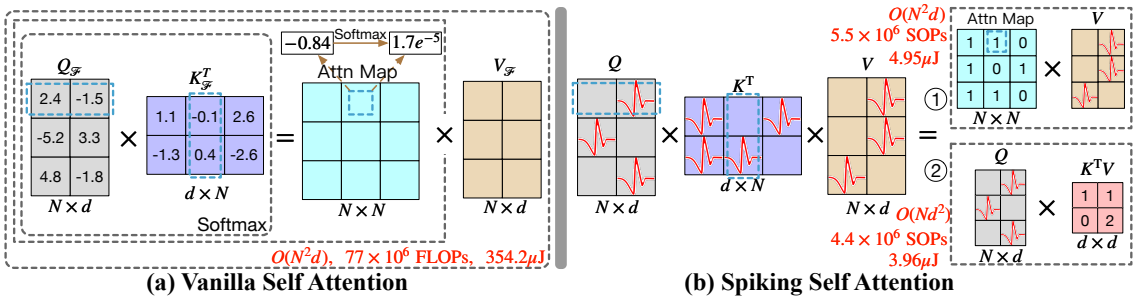
Unlike conventional ViTs, which operate on continuous-valued activations, SNN-ViTs process information through discrete spike events over time. This requires redesigning key Transformer components—including patch embedding, self-attention, and feed-forward layers—to be compatible with spike-based computation.

Several approaches have been proposed in the literature to bridge the gap between Vision Transformers and Spiking Neural Networks. One early direction focuses on ANN-to-SNN conversion for Transformer architectures, where a pre-trained Vision Transformer is transformed into a spiking equivalent by mapping continuous activations to firing rates. While this approach benefits from leveraging well-optimized ANN models, it inherits several limitations commonly observed in CNN-to-SNN conversion [14], [27]. In particular, the reliance on rate-based encoding often leads to long inference latency due to the need for multiple simulation time steps. Moreover, the complex structure of self-attention mechanisms introduces additional challenges, resulting in potential accuracy degradation when approximated in the spiking domain.

To overcome these limitations, more recent studies have shifted toward directly trained spiking Transformers, where models are trained end-to-end using surrogate gradient methods [20]. This paradigm enables the network to learn spike-based representations natively, without relying on intermediate ANN formulations. A representative example is SpikeFormer, which introduces a fully spike-driven Transformer architecture by replacing conventional activation functions with spiking neurons while preserving the overall Transformer structure [28]. Such approaches have demonstrated competitive performance on standard benchmarks, while significantly improving energy efficiency due to their event-driven computation.

In addition to these methods, hybrid architectures have been proposed to further balance efficiency and representational power. For instance, models such as SpikeAtConv [6] integrate convolutional layers with spiking attention mechanisms, allowing the network to capture both local spatial features and global contextual relationships. By combining the strengths of convolution and attention, these hybrid designs aim to reduce the computational overhead associated with pure Transformer-based models while maintaining strong feature extraction capability.

Despite these advancements, existing SNN-ViT approaches are predominantly evaluated on high-resolution datasets such as ImageNet [7], where abundant spatial information is available. Consequently, their effectiveness on low-resolution and noisy data—such as capacitive fingerprint images—remains largely unexplored. This limitation highlights an important research gap in the current literature.



(a) Vanilla Self Attention **(b) Spiking Self Attention**

Figure 2.8: Illustration of vanilla self-attention (VSA)[25] and Spiking Self Attention (SSA)[28]

To address this gap, this thesis adopts a SpikeFormer-based Spiking Vision Transformer as the core architecture for the CapFingerId task. Specifically, the proposed model largely follows the original SpikeFormer design [28], while introducing task-specific adaptations to better suit low-resolution capacitive data. In particular, the number of Spiking Transformer encoder blocks and the feature embedding dimensions are carefully adjusted to balance representational capacity and computational efficiency under resource-constrained settings.

By retaining the core spike-driven self-attention mechanism and direct training strategy with surrogate gradients, the model preserves the advantages of SpikeFormer in capturing global dependencies. At the same time, the architectural scaling enables more efficient processing of small, noisy inputs, making it more suitable for capacitive fingerprint classification. This design choice allows the thesis to focus on evaluating the applicability and effectiveness of Spiking Vision Transformers in a new problem domain, rather than proposing a fundamentally new architecture.

CHAPTER 3. METHODOLOGY

This chapter presents two Spiking Neural Network (SNN)-based architectures for capacitive image-based fingerprint recognition. The first model is a RANC-based SNN designed as a fully spiking architecture trained entirely within the spiking domain and optimized for efficient neuromorphic hardware deployment. The proposed model utilizes burst encoding, overlapping segmentation, and Tea layers to extract spatio-temporal features from capacitive images. The second model is a Spiking Vision Transformer (Spiking ViT) based on the SpikeFormer framework, which incorporates spike-driven self-attention mechanisms to capture global dependencies in the input data. In addition, the chapter introduces the Spiking Patch Splitting (SPS) module and the Spiking Self-Attention (SSA) mechanism used in the transformer architecture. The input encoding process, spike aggregation strategy, and training methodology for both models are also described. Overall, the presented architectures aim to balance recognition performance, computational efficiency, and compatibility with neuromorphic computing systems.

3.1 Spiking Neural Network based on RANC Architecture

3.1.1 Design Overview

This section presents the overall design of the proposed Spiking Neural Network (SNN) for capacitive image-based fingerprint recognition. The model is developed to exploit the event-driven nature of spiking computation while maintaining competitive recognition performance compared to conventional deep learning approaches.

The input to the network is a preprocessed capacitive image of size 8×8, representing the spatial distribution of finger contact on the touchscreen sensor. To enable processing within the spiking domain, the static input is first transformed into a temporal spike representation using a burst encoding scheme over a fixed time window. This encoding allows the model to capture both spatial and temporal characteristics of the input signal.

The overal architecture follows a multi-stage processing pipeline. First, the encoded input is converted into a one-dimensional representation and partitioned into multiple overlapping segments. These segments are processed in parallel by independent spiking layers, enabling the network to extract localized features from different regions of the input. The use of overlapping segments ensures that important spatial patterns are preserved across segment boundaries and improves robustness to noise and variability in the capacitive images.

Next, the outputs from the parallel spiking layers are aggregated and further transformed by an additional spiking layer to produce a compact and discriminative feature representation. Finally, a pooling and classification stage is applied to generate the predicted finger identity.

The design of the proposed SNN is guided by three key principles. First, temporal encoding is employed to convert static inputs into spike trains, allowing the network to leverage the inherent advantages of spiking computation. Second, parallel processing with overlapping segmentation is introduced to enhance feature extraction capability while maintaining a lightweight structure. Third, hierarchical aggregation is used to progressively refine features and produce a robust representation for classification.

Overall, the proposed architecture provides a balance between computational efficiency and recognition performance, making it suitable for deployment in resource-constrained and energy-sensitive environments.

3.1.2 Input Encoding Scheme

To enable processing of static capacitive images within the spiking domain, the input is converted into spike trains using a burst encoding scheme. In this approach, each pixel intensity is represented by a short burst of spikes, where stronger intensities generate a higher number of spikes within a fixed temporal window. This encoding allows the model to capture both the magnitude and temporal characteristics of the input while remaining compatible with event-driven neuromorphic computation.

Mathematical Formulation Let $x_i \in [0, 1]$ denote the normalized intensity of the i-th pixel, and let T denote the number of discrete time steps (i.e., the encoding window or number of bins). The burst encoding scheme maps each pixel value to a binary spike train $S_i(t)$ of length T, where the number of spikes is proportional to the input intensity.

Specifically, the number of spikes generated for each pixel is given by:

$$n_i = \lfloor T \cdot x_i \rceil$$

where $\lfloor \cdot \rceil$ denotes rounding to the nearest integer.

The corresponding spike train is then defined as:

$$S_i(t) = \begin{cases} 1, & \text{if } t < n_i \\ 0, & \text{otherwise} \end{cases} \qquad \text{for } t = 0, 1, \ldots, T-1$$

This formulation produces a deterministic burst of spikes occurring at the beginning of the time window, where higher input intensities result in a greater number of spikes within the burst. The encoding is applied independently to each pixel, resulting in a spatiotemporal spike tensor of shape $T \times H \times W$ for each input sample.

Illustrative Examples To clarify the encoding process, consider a pixel with normalized intensity $x = 0.6$.

- For $T = 1$, the number of spikes is $n = \lfloor 1 \cdot 0.6 \rceil = 1$, resulting in the spike train:

  $$S(t) = [1]$$

  which corresponds to a binary threshold representation.

- For $T = 4$, the number of spikes is $n = \lfloor 4 \cdot 0.6 \rceil = 2$, resulting in the spike train:

  $$S(t) = [1,\ 1,\ 0,\ 0]$$

  which forms a burst of two spikes at the beginning of the time window.

These examples illustrate how increasing the number of time steps allows for a finer-grained representation of input intensity while preserving the burst structure.

Effect of Encoding Window Size An important parameter in the proposed encoding scheme is the number of time steps T, which determines the temporal resolution of the spike representation. In this work, T is varied from 1 to 16 to investigate its impact on recognition performance. When $T = 1$, the encoding reduces to a binary representation, where each pixel is mapped to a single spike or no spike, resulting in minimal temporal information. As T increases, the number of possible spike levels grows, enabling a finer-grained representation of input intensity through longer spike bursts.

This increase in temporal resolution allows the network to capture more detailed variations in the input signal, which is expected to improve feature discrimination and classification accuracy. However, larger values of T also introduce higher computational cost and longer inference latency. Therefore, evaluating different values

of T provides insight into the trade-off between accuracy and efficiency.

In the experimental section, i systematically analyze the performance of the proposed model under different encoding window sizes, demonstrating how the choice of T influences the overall effectiveness of the system.

3.1.3 Network Architecture Design

a, Parallel Segmentation Strategy

To enhance both computational efficiency and feature extraction capability, the input vector is partitioned into multiple overlapping segments that can be processed in parallel. Specifically, the 64-dimensional input vector is divided into seven segments, each consisting of 16 elements, with an overlap of 8 elements between consecutive segments (e.g., indices 0–15, 8–23, 16–31, and so on).

This design enables simultaneous processing across multiple spiking layers, significantly reducing the effective computation time compared to a fully sequential architecture. Since each segment is handled independently, the network can exploit parallelism at the layer level, which is particularly beneficial in neuromorphic systems where parallel spike processing is inherently supported.

In addition to improving computational speed, the overlapping segmentation strategy preserves continuity of spatial information across segment boundaries. This ensures that important local patterns are not lost and allows the model to maintain robustness against noise and variations in the capacitive images. As a result, the proposed approach achieves a balance between efficient computation and effective feature representation.

b, Spiking Feature Extraction

Spiking feature extraction in the proposed architecture is performed using the *Tea layer*, which serves as the fundamental computational unit of the network. Each *Tea layer* implements a hardware-constrained spiking transformation that approximates neuromorphic computation while remaining trainable using gradient-based methods.

Let $\mathbf{x} \in \{0,1\}^N$ denote the input spike vector after encoding. The *Tea layer* maintains two sets of parameters: a static weight matrix $\mathbf{W} \in \{-1,1\}^{N\times M}$ and a trainable connection matrix $\mathbf{C} \in [0,1]^{N\times M}$. During the forward pass, the connection matrix is discretized into a binary mask:

$$\hat{\mathbf{C}} = \text{round}(\mathbf{C}), \quad \hat{\mathbf{C}} \in \{0,1\}^{N\times M}$$

The effective synaptic matrix is then computed as:

$$\mathbf{S} = \mathbf{W} \odot \hat{\mathbf{C}}$$

where $\odot$ denotes element-wise multiplication. The membrane potential $\mathbf{u} \in \mathbb{R}^M$ of the neurons is computed as:

$$\mathbf{u} = \mathbf{x} \cdot \mathbf{S} + \mathbf{b}$$

where $\mathbf{b}$ is the bias vector (optionally applied and similarly discretized).

To produce the output spikes, a thresholding operation is applied. During inference, this corresponds to a hard spiking function:

$$\mathbf{y} = H(\mathbf{u}) = \begin{cases} 1, & \text{if } u_i \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

where $H(\cdot)$ is the Heaviside step function applied element-wise. During training, this non-differentiable operation is approximated using a sigmoid activation function:

$$\mathbf{y} = \sigma(\mathbf{u}) = \frac{1}{1 + e^{-\mathbf{u}}}$$

to enable gradient-based optimization. A custom gradient is used for the rounding operation to allow gradients to pass through unchanged during backpropagation.

Each input segment is processed independently by a dedicated *Tea layer*, allowing the network to extract localized spiking features in parallel. The resulting output vectors encode the spatio-temporal firing behavior of neurons and serve as high-level representations for subsequent aggregation and classification.

This formulation enables efficient computation using binary weights and sparse activations, while preserving the ability to train the network using standard deep learning techniques.

c, Spike-Based Aggregation and Classification

After the final spiking layer produces its output, the resulting spike activity must be aggregated to generate a classification decision. Unlike conventional neural networks, where outputs are typically continuous values, the proposed SNN produces

binary spike events over time. Therefore, a dedicated aggregation mechanism is required to convert temporal spike patterns into class predictions.

Let $\mathbf{Z} \in \{0,1\}^{T \times N}$ denote the output spike matrix of the final layer, where T is the number of time steps (ticks) and N is the number of neurons. The first step of aggregation is to accumulate spike activity over time:

$$\mathbf{s} = \sum_{t=1}^{T} \mathbf{Z}_t$$

where $\mathbf{s} \in \mathbb{R}^N$ represents the total spike count of each neuron.

The neurons are organized into groups corresponding to different classes. Specifically, the N neurons are evenly divided into K groups, where K is the number of classes. Each group contains $\frac{N}{K}$ neurons, and each neuron contributes to the prediction of a specific class.

The class-wise aggregation is then computed by summing the spike counts within each group:

$$y_k = \sum_{i \in \mathcal{G}_k} s_i, \quad k = 1, \ldots, K$$

where $\mathcal{G}_k$ denotes the set of neuron indices associated with class k. The resulting vector $\mathbf{y} \in \mathbb{R}^K$ represents the aggregated evidence for each class.

Optionally, a normalization step can be applied to scale the outputs and stabilize the prediction values. Finally, the class probabilities are obtained using a softmax function:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{y})$$

This additive pooling mechanism effectively transforms distributed spiking activity into a compact and interpretable classification output. By aggregating spikes both temporally and across neuron groups, the model leverages the collective behavior of neurons to make robust predictions.

d, Architecture Parameter Configuration

The performance and efficiency of the proposed SNN architecture are influenced by several key design parameters, including the segment size (*Elems*), the overlap between segments (*Overlap*), and the number of output neurons in each

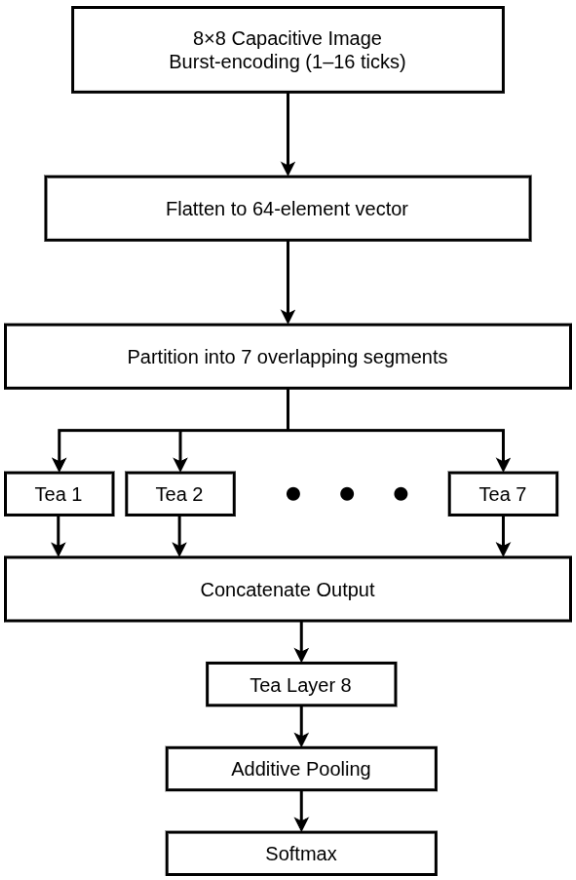


Figure 3.1: RANC-based SNN Model architecture

Tea layer (*Tea Out*). The configuration of these parameters follows the settings reported in Table 1 of the original work, which provide a suitable balance between recognition accuracy and computational cost.

The *Elems* parameter determines the number of elements in each input segment. This parameter controls the receptive field of each parallel processing branch. A smaller segment size allows the model to focus on fine-grained local patterns, while a larger segment size captures broader spatial context. In this work, a moderate segment size is selected to balance these two aspects, enabling effective local feature extraction without losing important contextual information.

The *Overlap* parameter specifies the number of shared elements between adjacent segments. Introducing overlap ensures that spatial continuity is preserved across segment boundaries, preventing the loss of critical information at segment edges. Additionally, overlapping segments introduce a degree of redundancy, which improves robustness to noise and variability in the capacitive fingerprint data.

The *Tea Out* parameter defines the number of output neurons in each Tea layer, and thus directly affects the representational capacity of the network. Increasing the number of neurons allows the model to learn more complex feature representations,

but also leads to higher computational cost. The selected value provides a trade-off between model expressiveness and efficiency, consistent with the design goals of neuromorphic systems.

Overall, these parameters jointly control the balance between local feature extraction, global context integration, and computational efficiency. The adopted configuration follows the empirical design choices presented in the reference architecture, ensuring consistency in both implementation and evaluation.

3.1.4 Training Strategy

a, Training Setup

The proposed SNN model is trained using the Adam optimizer with an initial learning rate of 0.001. The training process is conducted for a maximum of 300 epochs.

To prevent overfitting and reduce unnecessary computation, early stopping is applied with a patience of 10 epochs, monitoring the validation loss. This allows the training process to terminate once the model performance no longer improves.

b, Loss Function

The model is trained as a multi-class classification problem using the categorical cross-entropy loss function. The predicted outputs are obtained after the additive pooling layer followed by a softmax activation, producing a probability distribution over all target classes.

c, Training Mechanism of Tea Layers

During training, the Tea layers operate using continuous approximations to enable gradient-based optimization. Although the forward pass involves discrete operations such as rounding connections and thresholding neuron outputs, these non-differentiable functions are approximated during training.

Specifically, the spike generation function is replaced by a sigmoid activation to allow gradient propagation. Similarly, connection values are maintained as continuous parameters during training, while being discretized during inference to reflect the intended spiking behavior.

This approach allows the model to be trained using standard backpropagation while ensuring compatibility with discrete spiking computation during deployment.

d, Regularization

To improve generalization performance, dropout is applied after each Tea layer with a rate of 0.15. This helps mitigate overfitting, especially given the limited size and variability of the dataset.

3.1.5 Design Rationale and Advantages

The design of the proposed SNN architecture is guided by several key considerations to balance recognition performance and computational efficiency.

First, the use of *overlapping segmentation* combined with parallel processing enables efficient feature extraction from localized regions of the input. By dividing the input into multiple segments and processing them simultaneously, the model reduces effective computation time while preserving important spatial information across segment boundaries. The choice of segmentation parameters, including the segment size (*Elems*) and the degree of overlap (*Overlap*), plays a crucial role in this design. A moderate segment size allows the model to focus on meaningful local patterns, while overlapping segments ensure continuity of spatial information and improve robustness to noise and variations in the input.

Second, the adoption of *Tea layers* allows the network to operate with binary synaptic weights and discretized connections, making it suitable for neuromorphic implementations. This design reduces computational complexity and enables efficient simulation of spiking behavior while maintaining trainability. In addition, the number of output neurons in each Tea layer (*Tea Out*) determines the representational capacity of the network, providing a balance between expressive power and computational cost.

Third, the hierarchical structure of the network, consisting of *parallel feature extraction* followed by global aggregation, enables the model to capture both local and global patterns. This improves the discriminative capability of the network for fingerprint recognition tasks.

Finally, the use of *Additive Pooling* for output aggregation provides a robust mechanism for decision making based on spike activity. By aggregating neuron responses over time and across groups, the model leverages collective spiking behavior to produce stable and reliable predictions.

Overall, these design choices, together with carefully selected architectural parameters, contribute to an efficient and effective SNN architecture tailored for capacitive fingerprint recognition. The effects of these architectural parameters on model performance are further investigated in Chapter 4.

3.2 Spiking Vision Transformer (Spiking ViT)

While the RANC-based Spiking Neural Network in Section 3.1 efficiently captures local spatio-temporal features through event-driven computation, it lacks an explicit mechanism to model global dependencies across the input space [8]. This limitation is particularly critical for capacitive fingerprint images, where discriminative patterns are often spatially distributed and affected by noise and variability [1]. Vision Transformers address this issue via self-attention mechanisms, but their high computational cost limits their applicability in resource-constrained environments [25]. To overcome these challenges, this work adopts the SpikeFormer architecture, which integrates self-attention into the spiking domain to enable global feature modeling while maintaining energy efficiency [28].

3.2.1 Overview of SpikeFormer Architecture

The overall architecture of SpikeFormer follows a Transformer-style pipeline adapted to spike-based computation [28]. Given an input spatio-temporal representation $I \in \mathbb{R}^{T\times C\times H\times W}$, where T, C, H, and W denote the number of time steps, channels, height, and width respectively, the model processes the data through a sequence of spike-driven modules to produce the final prediction.

First, the input is passed through a Spiking Patch Splitting (SPS) module, which projects the input into a D-dimensional embedding space and partitions it into a sequence of N spike-form patches:

$$x = \mathrm{SPS}(I), \quad x \in \mathbb{R}^{T\times N\times D}. \tag{3.1}$$

To incorporate positional information in the absence of continuous-valued embeddings, SpikeFormer employs a conditional relative position encoding mechanism. Specifically, a convolutional layer followed by batch normalization and a spiking neuron layer is used to generate spike-form relative positional embeddings (RPE):

$$\mathrm{RPE} = \mathcal{SN}(\mathrm{BN}(\mathrm{Conv2d}(x))), \quad \mathrm{RPE} \in \mathbb{R}^{T\times N\times D}. \tag{3.2}$$

The positional information is then added to the patch embeddings:

$$X_0 = x + \mathrm{RPE}. \tag{3.3}$$

The resulting sequence is processed by a stack of L SpikeFormer encoder blocks. Each block consists of a Spiking Self-Attention (SSA) module followed by a spik-

ing multi-layer perceptron (MLP), both equipped with residual connections:

$$X'_l = \mathrm{SSA}(X_{l-1}) + X_{l-1}, \tag{3.4}$$

$$X_l = \mathrm{MLP}(X'_l) + X'_l, \quad l = 1, \ldots, L. \tag{3.5}$$

Unlike conventional self-attention, the SSA module operates directly on spike-form Query, Key, and Value representations without relying on softmax normalization, enabling efficient event-driven computation.

Finally, a global average pooling (GAP) layer aggregates the spatio-temporal features, and the resulting representation is fed into a fully connected classification head to produce the output prediction:

$$Y = \mathrm{CH}(\mathrm{GAP}(X_L)). \tag{3.6}$$

By combining spike-based patch embedding, efficient spiking self-attention, and hierarchical feature processing, SpikeFormer preserves the representational power of Transformers while significantly improving computational efficiency in spiking neural network settings.

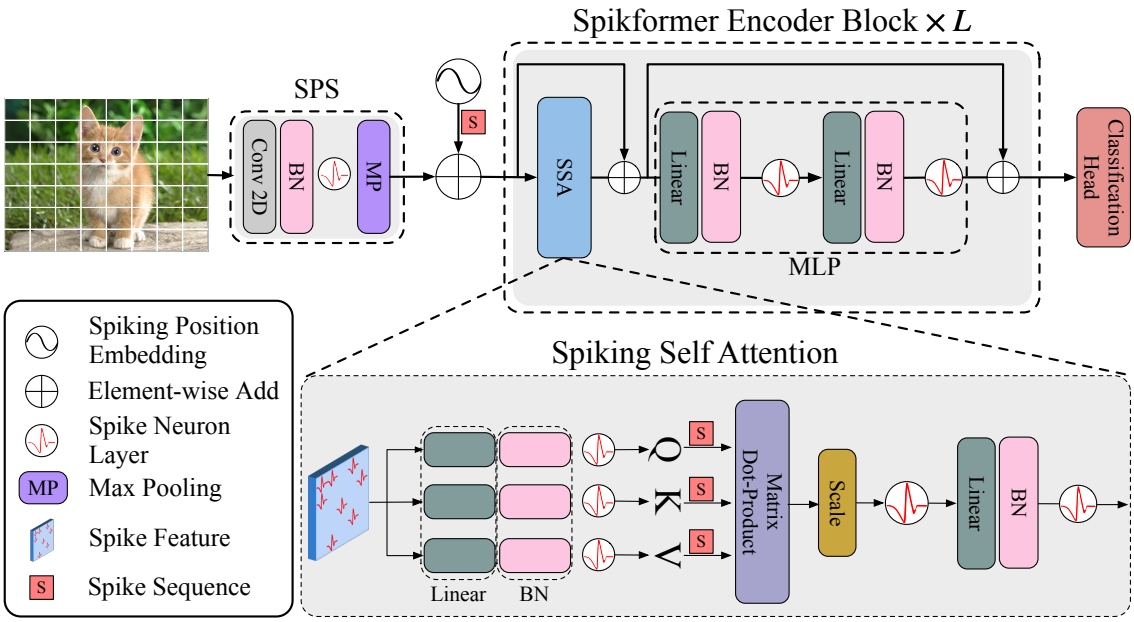


Figure 3.2: Overview of SpikeFormer architecture. The model consists of a Spiking Patch Splitting (SPS) module for input embedding, followed by a stack of SpikeFormer encoder blocks that utilize Spiking Self-Attention (SSA) and spiking MLPs. Finally, a global average pooling layer and classification head produce the output prediction.

3.2.2 Spiking Patch Splitting (SPS)

The Spiking Patch Splitting (SPS) module is responsible for transforming the input spike-based image representation into a sequence of embedded patches suitable for Transformer processing [28]. Given an input tensor $I \in \mathbb{R}^{T\times C\times H\times W}$, SPS projects the input into a D-dimensional spike-form feature space while simultaneously partitioning it into patch tokens.

Each SPS block consists of a sequence of operations including a 2D convolution layer, batch normalization, a spiking neuron activation, and a max-pooling layer:

$$x = \mathscr{M}\mathscr{P}\left(\mathcal{SN}(\mathrm{BN}(\mathrm{Conv2d}(I)))\right), \tag{3.7}$$

where $\mathrm{Conv2d}$ denotes a convolutional layer with kernel size 3×3 and stride 1, BN represents batch normalization, $\mathcal{SN}$ is the spiking neuron function, and $\mathscr{M}\mathscr{P}$ denotes max-pooling for spatial downsampling.

In this work, the spiking neuron function $\mathcal{SN}$ is implemented using a multi-step Leaky Integrate-and-Fire (LIF) model, specifically the `MultiStepLIFNode` provided by the SpikingJelly [29] framework, which integrates input signals over time and generates binary spike outputs in a vectorized manner.

Similar to the convolutional stem in Vision Transformers [30], [31], the inclusion of convolutional layers introduces inductive bias into the model, improving its ability to capture local spatial structures. Multiple SPS blocks can be stacked to progressively refine feature representations. In such cases, the number of output channels is gradually increased across blocks to match the desired embedding dimension D. For example, with a four-block SPS design, the channel dimensions can be configured as $D/8$, $D/4$, $D/2$, and D, respectively.

After passing through the SPS module, the input is transformed into a sequence of N spike-form patches:

$$x \in \mathbb{R}^{T\times N\times D}, \tag{3.8}$$

which serves as the input to the subsequent Spiking Transformer encoder. This design enables efficient feature extraction while preserving compatibility with spike-based computation.

3.2.3 Spiking Self-Attention Mechanism

The conventional self-attention mechanism is highly effective in modeling global dependencies but relies on dense floating-point operations and softmax normaliza-

tion, which are not well-suited for spiking neural networks. To address this limitation, this work adopts the Spiking Self-Attention (SSA) mechanism proposed in SpikeFormer [28], which reformulates attention into a spike-driven computation paradigm.

For completeness, the standard self-attention is defined as:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V, \tag{3.9}$$

where Q, K, and V denote the query, key, and value representations.

In the spiking formulation, the input feature X is first projected into query, key, and value spaces using learnable weight matrices, followed by batch normalization and spiking neuron activation:

$$Q = \mathcal{SN}_Q(\text{BN}(XW_Q)), \quad K = \mathcal{SN}_K(\text{BN}(XW_K)), \quad V = \mathcal{SN}_V(\text{BN}(XW_V)), \tag{3.10}$$

where $Q, K, V \in \mathbb{R}^{T\times N\times D}$ are spike-based tensors consisting of binary values. The spiking neuron function $\mathcal{SN}$ is implemented using a multi-step Leaky Integrate-and-Fire (LIF) model, ensuring that the representations evolve over time and remain compatible with event-driven computation.

Based on these spike-form representations, the attention operation is reformulated to operate directly on binary queries and keys. A scaling factor s is introduced to stabilize the magnitude of the matrix multiplication, leading to the intermediate formulation:

$$\text{SSA}'(Q, K, V) = \mathcal{SN}\left(QK^{\text{T}}V \cdot s\right). \tag{3.11}$$

The output is then further processed through a linear transformation and normalization:

$$\text{SSA}(Q, K, V) = \mathcal{SN}\left(\text{BN}(\text{Linear}(\text{SSA}'(Q, K, V)))\right). \tag{3.12}$$

A key distinction from conventional self-attention is the removal of the softmax operation. In standard attention, softmax ensures that the attention map is non-negative and normalized. However, in the spiking formulation, both Q and K are binary and inherently non-negative, resulting in a non-negative attention map QK^{T} without requiring additional normalization. Consequently, SSA directly aggregates

relevant spike-based features while suppressing irrelevant activations.

Moreover, the use of softmax is not only unnecessary but also impractical for spiking neural networks due to its reliance on continuous exponential operations. Compared to conventional attention operating on dense floating-point representations, the spike-based formulation is more aligned with the sparse and discrete nature of spiking signals. Since both the input X and the value V are in spike form, containing limited information, the use of conventional attention may introduce redundancy. In contrast, SSA provides a more suitable mechanism for modeling such spike-based representations.

Another important aspect of SSA is the incorporation of temporal dynamics. Specifically, all representations are defined over multiple time steps, i.e., $Q, K, V \in \mathbb{R}^{T\times N\times D}$. This allows the model to capture both spatial dependencies across patches and temporal patterns encoded in spike trains. A larger number of time steps T enables richer temporal representation and potentially improves discriminative performance, but also increases computational cost and latency. Conversely, a smaller T reduces computation and accelerates inference, at the expense of reduced temporal expressiveness. Therefore, selecting an appropriate value of T is crucial to balance performance and efficiency.

Within the SpikeFormer encoder, the SSA module is integrated with residual connections to stabilize training:

$$X_l' = \mathrm{SSA}(X_{l-1}) + X_{l-1}. \tag{3.13}$$

Overall, the spiking self-attention mechanism preserves the global modeling capability of Transformers while adapting it to spike-based computation, making it more suitable for spiking neural networks and neuromorphic systems.

CHAPTER 4. NUMERICAL RESULTS

This chapter presents the experimental evaluation of the proposed RANC-based Spiking Neural Network (SNN) and Spiking Vision Transformer (Spiking ViT) architectures for capacitive fingerprint recognition. The experiments investigate the influence of spatial segmentation, Tea layer output dimensionality, transformer depth, embedding dimension, and temporal encoding windows on both recognition performance and model complexity. For the RANC-based SNN, the results demonstrate that overlapping parallel segmentation and moderate temporal encoding significantly improve classification accuracy while maintaining lightweight computational complexity. In particular, the $(\mathit{Elems} = 16, \mathit{Overlap} = 8)$ configuration achieved the most favorable balance between recognition capability and parameter efficiency. The experiments further show that increasing the temporal encoding window improves spike-based feature representation, although the performance gain gradually saturates at larger timestep values. For the Spiking Vision Transformer, deeper encoder structures and larger embedding dimensions consistently improved classification performance through enhanced global feature modeling. Overall, the experimental findings validate the effectiveness of combining spiking computation, temporal encoding, and transformer-based attention mechanisms for efficient and accurate capacitive fingerprint recognition.

4.1 Experimental Evaluation of RANC-based SNN

4.1.1 Experimental Setup

This section presents the experimental configuration used to evaluate the proposed RANC-based Spiking Neural Network (SNN). The experiments were designed to investigate the effects of temporal encoding and architectural parameters on both recognition performance and model complexity.

a, Experimental Settings

The experimental analysis focused on two main groups of parameters:

- **Temporal encoding parameter**: the number of encoding time steps T, which determines the temporal resolution of the burst encoding scheme.
- **Parallel segmentation parameters**: including the segment size (*Elems*), overlap size (*Overlap*), and the number of output neurons in each Tea layer (*Tea Out*).

To systematically analyze the effect of each parameter, only one parameter group was varied at a time while the remaining parameters were fixed to their default

values.

The default configuration of the proposed architecture is summarized in Table 4.1.

Table 4.1: Default configuration of the proposed RANC-based SNN

Parameter	Value
Optimizer	Adam
Learning Rate	0.001
Maximum Epochs	300
Early Stopping Patience	10
Dropout Rate	0.15
Encoding Window T	4
Segment Size (Elems)	16
Overlap	8
Tea Out	256

4.1.2 Effect of Parallel Segmentation Strategy

This experiment investigates the effect of the proposed parallel segmentation strategy on the performance of the RANC-based SNN architecture. In particular, the analysis focuses on two important segmentation parameters: the segment size (*Elems*) and the overlap size (*Overlap*). The experiments were conducted using a temporal encoding window of $T = 1$, corresponding to the binary spike encoding setting.

The objective of this experiment is to evaluate how different segmentation configurations influence feature extraction capability, spatial continuity preservation, and model complexity. Different combinations of segment size and overlap were therefore explored while monitoring both classification performance and the number of trainable parameters.

Table 4.2 summarizes the experimental results obtained for different model configurations.

The experimental results demonstrate that the proposed parallel segmentation strategy significantly affects the recognition capability of the RANC-based SNN architecture across all fingerprint classification tasks. In general, configurations with moderate segment sizes and sufficient overlap consistently achieved better classification accuracy compared to configurations using either excessively large segments with limited overlap or very small fragmented segments.

Among all evaluated configurations, the $(16, 8)$ segmentation setting consistently produced the strongest overall performance. Under *Tea Out*=256, this con-

Table 4.2: Model configurations and performance under different parallel segmentation settings ($T = 1$).

Model			Index L/R	Thumb L/R	Thumb/Index	Thumb/Others	5 Fingers
Elems	Overlap	Tea Out					
16	8	64	0.60	0.84	0.78	0.81	0.42
32	8	64	0.56	0.77	0.75	0.77	0.39
16	4	32	0.57	0.77	0.74	0.76	0.40
32	16	128	0.61	0.84	0.79	0.82	0.43
16	4	64	0.59	0.84	0.77	0.80	0.41
32	16	64	0.57	0.81	0.76	0.79	0.40
16	8	128	0.62	0.85	**0.80**	0.83	0.41
32	8	256	0.61	0.84	0.79	0.82	0.43
8	2	64	0.57	0.81	0.75	0.78	0.40
8	2	256	0.60	0.84	0.78	0.81	0.42
32	8	128	0.58	0.82	0.76	0.79	0.41
8	4	64	0.59	0.83	0.77	0.80	0.41
16	4	256	0.62	**0.86**	0.81	**0.84**	0.42
8	4	32	0.56	0.76	0.74	0.76	0.39
32	16	256	0.62	**0.86**	**0.80**	0.83	0.42
16	8	32	0.57	0.80	0.75	0.78	0.40
32	8	32	0.55	0.62	0.74	0.76	0.39
8	2	32	0.56	0.79	0.75	0.77	0.40
32	16	32	0.55	0.66	0.74	0.76	0.39
8	4	256	0.61	0.85	**0.80**	**0.84**	0.43
8	2	128	0.58	0.82	0.76	0.79	0.41
16	4	128	0.61	0.85	0.79	0.82	0.43
16	**8**	**256**	**0.63**	**0.86**	0.79	0.82	**0.44**
8	4	128	0.60	0.85	0.78	0.81	0.42

figuration achieved the best results in several tasks, including 0.63 for *Index L/R*, 0.86 for *Thumb L/R*, and the highest 5-finger classification accuracy of 0.44, while using 59136 trainable parameters. Even with smaller output dimensions, the $(16, 8)$ setting remained highly stable, achieving 0.60, 0.62, and 0.63 for *Index L/R* under *Tea Out*=64, 128, and 256, respectively. These results indicate that the $(16, 8)$ configuration provides an effective balance between localized feature extraction and preservation of spatial continuity across neighboring fingerprint regions.

In contrast, configurations using larger segment sizes with insufficient overlap, particularly $(32, 8)$, consistently produced weaker performance. For example, the $(32, 8, 32)$ configuration achieved only 0.55 for *Index L/R*, 0.62 for *Thumb L/R*, 0.74 for *Thumb/Index*, and 0.39 for the 5-finger classification task, despite using only 4160 trainable parameters. Although increasing *Tea Out* improved performance for this segmentation setting, the overall accuracy remained lower than configurations with larger overlap. For instance, $(32, 8, 128)$ achieved 0.58 for *Index L/R* and 0.82 for *Thumb L/R*, which were still below the corresponding results obtained by the $(32, 16)$ configuration.

The importance of overlap can be observed clearly when comparing $(32, 8)$ and $(32, 16)$. Increasing the overlap from 8 to 16 improved *Thumb L/R* accuracy from

Table 4.3: Number of trainable parameters under different parallel segmentation configurations.

Model			#Params
Elems	Overlap	Tea Out	
16	8	64	14784
32	8	64	8320
16	4	32	5280
32	16	128	24960
16	4	64	10560
32	16	64	12480
16	8	128	29568
32	8	256	33280
8	2	64	10880
8	2	256	43520
32	8	128	16640
8	4	64	16320
16	4	256	42240
8	4	32	8160
32	16	256	49920
16	8	32	7392
32	8	32	4160
8	2	32	5440
32	16	32	6240
8	4	256	65280
8	2	128	21760
16	4	128	21120
16	8	256	59136
8	4	128	32640

0.77 to 0.81 for *Tea Out*=64, and from 0.82 to 0.84 for *Tea Out*=128. Similarly, *Index L/R* improved from 0.56 to 0.57 for *Tea Out*=64 and from 0.58 to 0.61 for *Tea Out*=128. The 5-finger classification accuracy also increased from 0.39 to 0.40 and 0.41 to 0.43, respectively. These improvements suggest that overlapping neighboring regions helps preserve shared spatial patterns and reduces information loss near segmentation boundaries.

Configurations using very small segments with limited overlap, such as $(8, 2)$, also exhibited reduced performance in several cases. Under *Tea Out*=32, the model achieved only 0.56 for *Index L/R*, 0.79 for *Thumb L/R*, and 0.40 for the 5-finger task, despite requiring 5440 parameters. Although increasing the output dimension to 256 improved the performance to 0.60, 0.84, and 0.42, the results remained slightly lower than those obtained by the $(16, 8)$ configuration. This behavior suggests that

excessive fragmentation of the input reduces the contextual information available to each processing branch, limiting the model's ability to learn discriminative fingerprint representations.

A similar overlap-related improvement can also be observed for smaller segmentation settings. Comparing $(8, 2)$ and $(8, 4)$, increasing the overlap improved *Thumb L/R* accuracy from 0.81 to 0.83 for *Tea Out*=64, and from 0.82 to 0.85 for *Tea Out*=128. The *Thumb/Others* classification accuracy also improved from 0.78 to 0.80 for *Tea Out*=64 and from 0.79 to 0.81 for *Tea Out*=128. These observations further confirm that overlapping segmentation contributes positively to feature continuity and classification robustness.

From a computational perspective, the segmentation strategy also strongly affects model complexity. Increasing overlap generally introduces additional trainable parameters because of the larger effective receptive regions processed by each Tea layer. For example, under *Tea Out*=128, increasing overlap from 8 to 16 for the 32-element segmentation setting increased the parameter count from 16640 to 24960. Similarly, for the 8-element segmentation setting, increasing overlap from 2 to 4 increased the number of parameters from 21760 to 32640. Moreover, increasing *Tea Out* substantially enlarged the model size, as observed in the $(16, 8)$ configuration where the parameter count increased from 7392 at *Tea Out*=32 to 59136 at *Tea Out*=256.

Overall, the experimental results demonstrate that the proposed overlapping parallel segmentation strategy effectively improves feature extraction capability and classification stability across multiple fingerprint classification tasks. Among all evaluated configurations, moderate segment sizes with moderate overlap, particularly the $(16, 8)$ configuration, provide the most favorable balance between recognition performance and computational complexity.

4.1.3 Effect of Tea Out

This experiment investigates the influence of the *Tea Out* parameter on the representational capability and classification performance of the proposed parallel RANC-based SNN architecture. The *Tea Out* parameter determines the dimensionality of the output feature representation generated by each Tea layer. Increasing this value enlarges the latent spiking feature space and allows the network to encode more discriminative fingerprint characteristics.

The experiments were conducted under multiple segmentation configurations while varying the *Tea Out* dimension among 32, 64, 128, and 256. The objective is to analyze the trade-off between classification performance and model complexity

as the output dimensionality increases.

The experimental results in Table 4.2 show that increasing the *Tea Out* dimension generally improves recognition performance across all evaluation tasks. In most configurations, larger output dimensions consistently achieved higher scores for *Index L/R*, *Thumb L/R*, *Thumb/Index*, *Thumb/Others*, and *5 Fingers* classification tasks.

For the $(16, 8)$ segmentation configuration, increasing *Tea Out* from 32 to 64, 128, and 256 improved the *Index L/R* score from 0.57 to 0.60, 0.62, and 0.63, respectively. Similarly, the *Thumb L/R* performance increased from 0.80 to 0.84, 0.85, and finally 0.86. The *5 Fingers* evaluation also improved from 0.40 to 0.42, 0.41, and 0.44. These results indicate that larger Tea representations enable the model to capture richer local fingerprint structures and improve classification robustness.

A similar trend can also be observed for the $(32, 16)$ configuration. When *Tea Out*=32, the model achieved relatively low scores, including 0.55 for *Index L/R* and 0.66 for *Thumb L/R*. Increasing the output dimension to 64, 128, and 256 progressively improved the performance to 0.57, 0.61, and 0.62 for *Index L/R*, while *Thumb L/R* improved to 0.81, 0.84, and 0.86, respectively. The *Thumb/Others* metric also improved from 0.76 to 0.79, 0.82, and 0.83.

For smaller segmentation settings such as $(8, 2)$, the same behavior was consistently observed. Increasing *Tea Out* from 32 to 64, 128, and 256 improved the *Thumb L/R* score from 0.79 to 0.81, 0.82, and 0.84, respectively. Likewise, the *Thumb/Index* metric improved from 0.75 to 0.75, 0.76, and 0.78. Although the improvement becomes smaller at larger output dimensions, higher-dimensional Tea representations still provide more stable classification performance across multiple evaluation tasks.

The results suggest that increasing the output dimensionality enables each Tea branch to encode richer localized spiking features. Since the proposed architecture relies on parallel feature extraction from segmented fingerprint regions, larger feature representations provide additional flexibility for capturing subtle ridge patterns and spatial variations in capacitive fingerprint images.

However, increasing the *Tea Out* dimension also substantially increases the number of trainable parameters, as shown in Table 4.3. For the $(16, 8)$ configuration, the parameter count increased from 7392 at *Tea Out*=32 to 14784, 29568, and 59136 for output dimensions of 64, 128, and 256, respectively. Similarly, under the $(8, 4)$ configuration, the number of parameters increased from 8160 to 16320, 32640, and 65280.

Although larger output dimensions generally improve performance, the improvement becomes progressively smaller beyond *Tea Out*=128. For example, under the $(16, 8)$ configuration, increasing *Tea Out* from 128 to 256 improved the *Index L/R* score by only 0.01 ($0.62 \rightarrow 0.63$) and the *Thumb L/R* score by only 0.01 ($0.85 \rightarrow 0.86$), while the number of trainable parameters doubled from 29568 to 59136. Similar diminishing returns can also be observed in the $(32, 16)$ and $(8, 2)$ configurations.

Overall, the experimental results demonstrate that the *Tea Out* parameter plays a critical role in balancing representational capability and computational complexity. Small output dimensions often lead to insufficient feature representation and reduced classification performance, while excessively large output dimensions significantly increase model complexity with relatively limited performance gain. Among the evaluated settings, moderate-to-large output dimensions such as 128 and 256 provide the most favorable balance between recognition accuracy and computational efficiency for the proposed parallel RANC-based SNN architecture.

4.1.4 Effect of Temporal Encoding Window

This experiment investigates the effect of the temporal encoding window T on the performance of the proposed RANC-based SNN architecture. To isolate the influence of temporal encoding, the experiments were conducted using the best-performing segmentation configuration identified in the previous experiments, namely *Elems*=16, *Overlap*=8, and *Tea Out*=256.

The temporal encoding window determines the number of discrete time steps used in the burst encoding process. Increasing T allows each pixel intensity to be represented with finer temporal resolution through longer spike bursts, potentially enabling the network to capture richer spatio-temporal information.

Table 4.4 summarizes the experimental results obtained under different temporal encoding windows.

Table 4.4: Performance of the proposed RANC-based SNN under different temporal encoding windows using the best segmentation configuration (*Elems*=16, *Overlap*=8, *Tea Out*=256).

Time Steps (T)	Index L/R	Thumb L/R	Thumb/Index	Thumb/Others	5 Fingers
1	0.63	0.86	0.79	0.82	0.44
2	0.63	0.87	0.80	0.83	0.45
4	0.64	0.87	0.81	0.85	0.45
8	0.65	0.89	0.82	0.86	0.46

This experiment investigates the influence of the temporal encoding window on

the recognition performance of the proposed RANC-based SNN architecture. The temporal encoding window, represented by the number of timesteps T, determines how many temporal spike events are generated for each input sample during inference. Increasing the number of timesteps enables the network to accumulate richer temporal information and capture more discriminative spiking representations.

The experiments were conducted using the best-performing segmentation configuration identified in previous experiments, namely ($\textit{Elems} = 16, \textit{Overlap} = 8, \textit{Tea Out} = 256$). The timestep values were varied from $T = 1$ to $T = 8$ in order to analyze the trade-off between recognition capability and temporal computational complexity.

The experimental results in Table 4.4 demonstrate that increasing the temporal encoding window generally improves the classification performance across all evaluation tasks. In particular, larger timestep values consistently produced higher recognition scores for *Index L/R*, *Thumb L/R*, *Thumb/Index*, *Thumb/Others*, and *5 Fingers* classification tasks.

When using a single timestep ($T = 1$), corresponding to binary spike encoding without temporal accumulation, the model achieved scores of 0.63 for *Index L/R*, 0.86 for *Thumb L/R*, 0.79 for *Thumb/Index*, 0.82 for *Thumb/Others*, and 0.44 for the *5 Fingers* task. These results indicate that even a single-step spiking representation is capable of learning meaningful fingerprint features.

Increasing the temporal window to $T = 2$ produced small but consistent improvements across multiple tasks. The *Thumb L/R* score increased from 0.86 to 0.87, while *Thumb/Index* improved from 0.79 to 0.80, and *Thumb/Others* improved from 0.82 to 0.83. The *5 Fingers* score also increased slightly from 0.44 to 0.45. These improvements suggest that introducing additional temporal spike events allows the network to construct richer internal spiking representations.

More noticeable performance improvement was observed when increasing the timestep value to $T = 4$. Under this configuration, the *Index L/R* score improved to 0.64, while *Thumb/Index* and *Thumb/Others* increased to 0.81 and 0.85, respectively. The results indicate that moderate temporal windows enable the proposed SNN architecture to better capture subtle spatial and temporal variations in capacitive fingerprint patterns through burst-based spike encoding.

The best overall performance was obtained when using $T = 8$. Under this configuration, the model achieved 0.65 for *Index L/R*, 0.89 for *Thumb L/R*, 0.82 for *Thumb/Index*, 0.86 for *Thumb/Others*, and 0.46 for the *5 Fingers* task. Compared to the $T = 1$ configuration, the improvements demonstrate that longer temporal win-

dows enable the network to accumulate more stable and discriminative spike-based feature representations.

The selected timestep settings were motivated by commonly adopted temporal configurations in SpikeFormer [28] and related spiking transformer architectures, where small-to-moderate temporal windows such as $T = 1$, $T = 2$, $T = 4$, and $T = 8$ were shown to provide effective trade-offs between recognition performance and computational efficiency. Following this experimental convention enables more consistent comparison with existing spiking transformer literature while remaining suitable for lightweight neuromorphic computation.

Although increasing the timestep value generally improves performance, larger temporal windows also introduce additional computational overhead and inference latency because spike-based operations must be performed repeatedly across multiple timesteps. Moreover, the performance improvement gradually becomes smaller at higher timestep values. For example, increasing the timestep from $T = 4$ to $T = 8$ improved the *Thumb/Index* score by only 0.01 and the *5 Fingers* score by only 0.01. This behavior suggests diminishing representational benefits at very large temporal windows.

Overall, the experimental results demonstrate that temporal encoding plays an important role in improving the discriminative capability of the proposed SNN architecture. Moderate temporal windows such as $T = 4$ and $T = 8$ provide the most favorable balance between recognition performance and computational efficiency, while excessively large timestep values may introduce significantly higher computational cost with relatively limited additional performance improvement.

4.1.5 Discussion

The experimental results demonstrate that both spatial segmentation and temporal encoding significantly influence the performance of the proposed RANC-based SNN architecture. Among the evaluated segmentation settings, the $(16, 8)$ configuration consistently achieved the best balance between recognition accuracy and model complexity, indicating that moderate segment sizes with sufficient overlap are effective for preserving local spatial continuity. Increasing the *Tea Out* dimension generally improved classification performance by enabling richer spiking feature representations, although the improvement gradually saturated at higher output dimensions. In particular, increasing *Tea Out* from 128 to 256 produced only marginal performance gains while substantially increasing the number of trainable parameters.

The temporal encoding experiments further confirmed the importance of tem-

poral spike accumulation in enhancing discriminative capability. Increasing the timestep value from $T = 1$ to $T = 8$ consistently improved recognition performance across all evaluation tasks, demonstrating the effectiveness of burst-based spike encoding for capacitive fingerprint classification. However, the improvement became progressively smaller at larger timestep values, suggesting diminishing returns beyond moderate temporal resolutions. In addition, the proposed architecture maintained relatively lightweight computational complexity while achieving competitive recognition performance through binary spike representations, discretized Tea connections, and localized parallel processing. Overall, the experimental findings validate the effectiveness of combining overlapping parallel segmentation with temporal spike encoding to achieve an efficient balance between recognition capability, representational power, and neuromorphic computational efficiency.

4.2 Experimental Evaluation of Spiking Vision Transformer

4.2.1 Experimental Setup

This section presents the experimental configuration used to evaluate the proposed Spiking Vision Transformer architecture. Similar to the previous RANC-based experiments, the evaluation focuses on both recognition performance and model complexity under different architectural and temporal configurations.

The experiments investigate the influence of three primary parameter groups adopted from the original SpikeFormer design [28]:

- **Number of Spiking Transformer Encoder blocks** (L), which controls the network depth.
- **Feature embedding dimension** (D), which determines the representational capacity of the transformer features.
- **Temporal encoding window** (T), which controls the number of spike propagation timesteps.

To ensure fair comparison, only one parameter group was varied at a time while the remaining settings were fixed to their default configuration. The experiments were evaluated using the same fingerprint classification tasks and performance metrics described in the previous subsection, including *Index L/R*, *Thumb L/R*, *Thumb/Index*, *Thumb/Others*, and *5 Fingers* classification accuracy.

The default configuration of the proposed Spiking Vision Transformer is summarized in Table 4.5.

Table 4.5: Default configuration of the proposed Spiking Vision Transformer

Parameter	Value
Optimizer	Adam
Learning Rate	0.001
Maximum Epochs	300
Early Stopping Patience	10
Dropout Rate	0.15
Encoder Blocks (L)	8
Embedding Dimension (D)	512
Encoding Window (T)	4

4.2.2 Effect of Number of Encoder Blocks

Table 4.6 presents the experimental results obtained under different Spiking Vision Transformer configurations with varying encoder depths and embedding dimensions. The results demonstrate that increasing the number of encoder blocks generally improves the recognition capability of the proposed architecture by enabling deeper hierarchical spike-based feature extraction.

Compared with the baseline model, all Spiking Vision Transformer configurations achieved noticeable performance improvements across most classification tasks. In particular, the configuration using 8 encoder blocks and embedding dimension 384 improved the *Thumb L/R* accuracy from 0.90 to 0.92, while also improving the *Thumb/Index* accuracy from 0.84 to 0.86. These results indicate that the transformer-based spiking representation is capable of learning more discriminative spatial relationships than the baseline architecture.

When the embedding dimension was fixed at $D = 512$, increasing the number of encoder blocks from $L = 6$ to $L = 8$ consistently improved performance across all evaluation tasks. Specifically, *Index L/R* accuracy increased from 0.67 to 0.70, while *Thumb/Others* accuracy improved from 0.87 to 0.90. This behavior suggests that deeper transformer structures allow the network to capture richer contextual dependencies between fingerprint regions through multiple stages of spike-based self-attention.

Further increasing the encoder depth to $L = 10$ produced additional improvement in several tasks. Under the $(L = 10, D = 512)$ configuration, the model achieved the highest *Thumb/Others* accuracy of 0.92 and the best *5 Fingers* classification accuracy of 0.48. These results demonstrate that deeper spiking transformer architectures are particularly beneficial for more challenging multi-class classification scenarios requiring stronger global feature aggregation.

Table 4.6: Performance of Spiking Vision Transformer under different encoder depth and embedding dimension configurations ($T = 4$).

Architecture		Index L/R	Thumb L/R	Thumb/Index	Thumb/Others	5 Fingers
Encoder Blocks (L)	Embedding Dimension (D)					
Baseline Model		0.65	0.90	0.84	0.86	0.46
8	384	0.65	0.92	0.86	0.86	0.45
6	512	0.67	0.92	0.87	0.87	0.45
8	512	0.70	0.94	0.89	0.90	0.46
10	512	0.70	0.94	0.89	**0.92**	**0.48**
8	768	**0.71**	**0.95**	**0.90**	**0.92**	0.47

The results also indicate that the improvement gradually saturates at higher model capacities. Although the $L = 10$ configuration outperformed shallower models in some tasks, the performance gain compared with $L = 8$ remained relatively small. This suggests that excessively deep transformer architectures may introduce additional computational complexity while providing only marginal improvement in recognition capability.

Overall, the experiments confirm that encoder depth plays an important role in the representational power of the proposed Spiking Vision Transformer. Moderate-to-deep configurations, particularly those using 8 to 10 encoder blocks, provide the most favorable balance between classification performance and architectural complexity for capacitive fingerprint recognition.

4.2.3 Effect of Changing Embedding Dimensions

The experimental results further demonstrate that the embedding dimension significantly affects the representational capability of the proposed Spiking Vision Transformer. Increasing the embedding dimension generally improved classification performance across most evaluation tasks by allowing the network to encode richer spike-based feature representations.

When the number of encoder blocks was fixed at $L = 8$, increasing the embedding dimension from $D = 384$ to $D = 512$ consistently improved recognition accuracy. In particular, *Index L/R* accuracy increased from 0.65 to 0.70, while *Thumb/Others* accuracy improved from 0.86 to 0.90. Similarly, the *Thumb/Index* classification accuracy increased from 0.86 to 0.89. These improvements indicate that larger embedding spaces enable the transformer to preserve more discriminative spatial and temporal spike features.

Further increasing the embedding dimension to $D = 768$ produced the best overall performance in several classification tasks. Under the $(L = 8, D = 768)$ configuration, the model achieved the highest *Index L/R* accuracy of 0.71, the highest *Thumb L/R* accuracy of 0.95, and the best *Thumb/Index* accuracy of 0.90. These

Table 4.7: Performance of the Spiking Vision Transformer under Different Temporal Encoding Windows.

Time Step	**Index L/R**	**Thumb L/R**	**Thumb/Index**	**Thumb/Others**	**5 Fingers**
Baseline Model	0.65	0.90	0.84	0.86	0.46
1	0.65	0.89	0.86	0.86	0.45
2	0.67	0.92	0.87	0.86	0.46
4	0.70	0.95	0.89	0.89	0.48
8	**0.71**	**0.95**	**0.90**	**0.90**	**0.48**

results suggest that larger embedding dimensions improve the ability of the self-attention mechanism to model complex relationships between fingerprint regions.

However, the experimental results also indicate diminishing performance gains at very large embedding dimensions. Although the $D = 768$ configuration achieved slightly better performance than $D = 512$, the improvement remained relatively modest in several tasks. For example, the *Thumb/Others* accuracy remained unchanged at 0.92, while the *5 Fingers* accuracy improved only marginally from 0.46 to 0.47. This suggests that increasing embedding dimensionality beyond moderate values may introduce additional computational overhead with limited practical benefit.

Compared with the baseline model, all transformer-based configurations achieved substantially higher performance, demonstrating the effectiveness of spike-based transformer representations for capacitive fingerprint recognition. The results confirm that larger embedding dimensions improve feature diversity and contextual representation learning within the spiking self-attention framework.

Overall, the experiments show that embedding dimension is a critical factor influencing the representational capacity of the Spiking Vision Transformer. Moderate-to-large embedding dimensions, particularly $D = 512$ and $D = 768$, provide the most favorable balance between recognition performance and architectural efficiency.

4.2.4 Effect of Temporal Encoding Window

This experiment investigates the influence of temporal encoding on the proposed Spiking Vision Transformer. To ensure fair comparison across different temporal resolutions, all timestep experiments were conducted using the same transformer configuration with $L = 8$ encoder blocks and embedding dimension $D = 512$, while only the number of encoding timesteps was varied.

The experimental results demonstrate that increasing the temporal encoding window generally improves the recognition capability of the proposed architecture.

Compared with the baseline model, all spiking transformer configurations achieved competitive or superior performance across most evaluation tasks, indicating that temporal spike-based processing effectively enhances feature representation learning.

When using a single timestep ($T = 1$), the model achieved 0.89 accuracy for the *Thumb L/R* task and 0.86 for *Thumb/Index*. Although the performance remained comparable to the baseline model, the limited temporal resolution restricted the ability of the network to fully exploit temporal spike dynamics.

Increasing the timestep to $T = 2$ produced consistent improvements across several tasks. In particular, *Index L/R* accuracy improved from 0.65 to 0.67, while *Thumb L/R* increased from 0.89 to 0.92. These results suggest that introducing additional temporal information enables the transformer to learn richer spike-based contextual representations.

More significant improvement was observed when increasing the temporal window to $T = 4$. Under this configuration, the model achieved 0.70 accuracy for *Index L/R*, 0.95 for *Thumb L/R*, and 0.89 for both *Thumb/Index* and *Thumb/Others*. The *5 Fingers* classification accuracy also increased to 0.48, outperforming the baseline model. These results indicate that moderate temporal windows allow the spiking transformer to better capture temporal correlations and subtle fingerprint variations.

Further increasing the temporal window to $T = 8$ produced only marginal improvement. Although the model achieved the best overall performance, including 0.71 for *Index L/R* and 0.90 for both *Thumb/Index* and *Thumb/Others*, the improvement over $T = 4$ remained relatively small. This behavior suggests diminishing returns at higher timestep values, where additional temporal information contributes limited representational benefit.

Overall, the experiments confirm that temporal encoding plays a crucial role in the performance of the proposed Spiking Vision Transformer. Moderate timestep values such as $T = 4$ and $T = 8$ provide the most favorable balance between recognition accuracy and computational efficiency, while excessively small temporal windows limit the ability of the network to exploit spike-based temporal dynamics effectively.

4.2.5 Discussion

The experimental results demonstrate that the proposed Spiking Vision Transformer consistently outperforms the baseline model across most fingerprint clas-

sification tasks. The integration of spike-based self-attention enables the network to capture richer spatial and temporal relationships between fingerprint regions. Increasing the number of encoder blocks generally improves recognition performance, particularly for more complex tasks such as *Thumb/Others* and *5 Fingers*. However, the improvement gradually saturates at deeper configurations, indicating diminishing returns in performance compared with the additional computational cost.

Similarly, larger embedding dimensions improve the representational capability of the transformer by allowing more discriminative spike-based feature encoding. The best overall performance was achieved using moderate-to-large embedding dimensions such as $D = 512$ and $D = 768$. The temporal encoding experiments also confirm the importance of spike-based temporal dynamics. Increasing the timestep from $T = 1$ to $T = 4$ significantly improved classification accuracy, while further increasing to $T = 8$ produced only marginal gains. Overall, the results indicate that moderate encoder depth, embedding dimension, and temporal window size provide the best balance between recognition performance and computational efficiency for capacitive fingerprint recognition.

CHAPTER 5. CONCLUSIONS

5.1 Summary

This thesis investigated the development of energy-efficient deep learning models for capacitive image-based finger classification, with a particular emphasis on Spiking Neural Networks (SNNs) as a viable alternative to conventional convolutional and transformer-based approaches. The work was motivated by the growing need for context-aware, low-power sensing in mobile and wearable devices, where continuous finger-type recognition must operate within tight energy and computational budgets.

Two distinct SNN-based architectures were proposed and evaluated on the CapFingerId dataset. The first is a RANC-based SNN, designed as a fully spiking architecture compatible with neuromorphic hardware platforms. The model employs burst encoding to convert static capacitive images into spatiotemporal spike trains, processes the encoded input through parallel overlapping segments via Tea layers, and aggregates spiking activity using additive pooling for classification. Experimental evaluation demonstrated that the proposed segmentation strategy — particularly the (`Elems=16`, `Overlap=8`) configuration — achieves the most favorable balance between recognition accuracy and model parameter efficiency. The temporal encoding experiments further confirmed that moderate timestep values ($T = 4$ to $T = 8$) provide meaningful performance gains, while larger windows yield diminishing returns relative to their additional computational cost. Overall, the RANC-based SNN achieved competitive accuracy compared to the CNN baseline from the original CapFingerId study, while operating with significantly fewer parameters and a neuromorphic-compatible design.

The second proposed architecture is a Spiking Vision Transformer (Spiking ViT), built upon the SpikeFormer framework. This model integrates a Spiking Patch Splitting (SPS) module for feature embedding and a Spiking Self-Attention (SSA) mechanism to capture global spatial dependencies within the spike domain. By adapting the encoder depth and embedding dimensions to suit the low-resolution nature of capacitive data, the proposed Spiking ViT demonstrated consistent improvement over the CapFingerId baseline across all evaluated classification tasks. Notably, the best-performing configuration ($L = 8$, $D = 768$, $T = 4$) achieved a Thumb L/R accuracy of 0.95 and a Thumb/Index accuracy of 0.90, representing approximately 6% improvement over the baseline model. The experimental analysis showed that deeper encoder structures and larger embedding dimensions generally

improve performance, though gains saturate at high model capacities, reinforcing the importance of selecting a balanced architecture.

Together, the results validate that SNN-based architectures — particularly those incorporating transformer-level attention mechanisms — are capable of efficient and accurate finger-type recognition from capacitive images. The event-driven and sparse computation inherent to SNNs makes these models well-suited for deployment on edge devices where energy constraints are critical.

Despite these contributions, the work has several limitations that should be acknowledged. First, the actual deployment of the proposed RANC-based SNN on FPGA hardware was beyond the scope of this thesis; the energy efficiency gains from neuromorphic hardware therefore remain to be empirically verified. Second, the Spiking ViT adopts a hybrid CNN-SNN design in its SPS module, which introduces some non-spiking operations and partially limits the full spike-driven computation ideal for neuromorphic hardware. Third, the study focused exclusively on tap interactions from the CapFingerId dataset; generalization to dynamic gestures such as swipes and scrolls was not evaluated.

5.2 Suggestions for Future Work

Several directions are proposed to extend and strengthen the findings of this thesis.

The most immediate next step is the hardware deployment of the RANC-based SNN on a physical neuromorphic platform, such as an FPGA running the RANC architecture. This would enable direct measurement of energy-per-inference and latency, providing concrete evidence of the power efficiency advantages that are claimed theoretically in this work.

A second direction concerns the development of a fully spike-driven Vision Transformer, eliminating the convolutional components in the SPS module and replacing them with pure spiking operations. This would make the Spiking ViT more compatible with neuromorphic hardware and further reduce power consumption during inference.

Third, future work could explore the extension of both architectures to dynamic gesture inputs — including dragging and scrolling — which introduce additional temporal variability not addressed in this study. Encoding motion-dependent spike patterns could further improve context awareness in real-world interaction scenarios.

Finally, a systematic energy benchmarking study comparing the proposed SNN

models against CNN and standard ViT baselines in terms of millijoules per inference (mJ/inference) would significantly strengthen the practical case for neuromorphic finger recognition, closing an important gap identified in the current literature.

REFERENCE

[1] H. V. Le, S. Mayer **and** N. Henze, "Investigating the feasibility of finger identification on capacitive touchscreens using deep learning," **in** *Proceedings of the 24th International Conference on Intelligent User Interfaces*, **jourser** IUI '19, Marina del Ray, California: Association for Computing Machinery, 2019, 637–649.

[2] Z. Huang, C. Gao, H. Wang, X. Deng, Y.-K. Lai, C. Ma, S.-f. Qin, Y.-J. Liu **and** H. Wang, "SpeciFingers: Finger Identification and Error Correction on Capacitive Touchscreens," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, **jourvol** 8, **number** 1, **march** 2024.

[3] M. Voudaskas, J. I. MacLean, N. A. W. Dutton, B. D. Stewart **and** I. Gyongy, "Spiking Neural Networks in Imaging: A Review and Case Study," *Sensors*, **jourvol** 25, **number** 21, 2025.

[4] J. He, M. Tian, Y. Jiang, H. Wang, T. Wang, X. Zhou, L. Liu, N. Wu, Y. Wang **and** C. Shi, "A visual cortex-inspired edge neuromorphic hardware architecture with on-chip multi-layer STDP learning," *Computers and Electrical Engineering*, **jourvol** 120, **page** 109 806, 2024.

[5] R. Beaubois, J. Cheslet, Y. Ikeuchi, P. Branchereau **and** T. Levi, "Real-time multicompartment Hodgkin-Huxley neuron emulation on SoC FPGA," *Frontiers in Neuroscience*, **jourvol** Volume 18 - 2024, 2024.

[6] W. Liao **and** W. Wang, *SpikeAtConv: An Integrated Spiking-Convolutional Attention Architecture for Energy-Efficient Neuromorphic Vision Processing*, 2024.

[7] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg **and** L. Fei-Fei, *ImageNet Large Scale Visual Recognition Challenge*, 2015.

[8] J. Mack, R. Purdy, K. Rockowitz, M. Inouye, E. Richter, S. Valancius, N. Kumbhare, M. S. Hassan, K. Fair, J. Mixter **and** A. Akoglu, "RANC: Reconfigurable Architecture for Neuromorphic Computing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, **jourvol** 40, **number** 11, 2265–2278, **november** 2021.

[9] H. Nam, K.-H. Seol, J. Lee, H. Cho **and** S. W. Jung, "Review of Capacitive Touchscreen Technologies: Overview, Research Trends, and Machine Learning Approaches," *Sensors*, **jourvol** 21, **number** 14, 2021.

[10] Y. Yu, Q. Niu, X. Li, J. Xue, W. Liu **and** D. Lin, "A Review of Fingerprint Sensors: Mechanism, Characteristics, and Applications," *Micromachines*, **jourvol** 14, **number** 6, 2023.

[11] P. Panda **and** K. Roy, *Unsupervised Regenerative Learning of Hierarchical Features in Spiking Deep Networks for Object Recognition*, 2016.

[12] S. Park, S. Kim, H. Choe **and** S. Yoon, *Fast and Efficient Information Transmission with Burst Spikes in Deep Spiking Neural Networks*, 2019.

[13] W. Gerstner **and** W. M. Kistler, *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, 2002.

[14] P. U. Diehl **and** M. Cook, "Unsupervised learning of digit recognition using spike-timing-dependent plasticity," *Frontiers in Computational Neuroscience*, **jourvol** Volume 9 - 2015, 2015.

[15] S. J. Thorpe, A. Delorme **and** R. van Rullen, "Spike-based strategies for rapid processing," *Neural networks : the official journal of the International Neural Network Society*, **jourvol** 14 6-7, **pages** 715–25, 2001.

[16] T. Masquelier **and** S. J. Thorpe, "Unsupervised Learning of Visual Features through Spike Timing Dependent Plasticity," *PLOS Computational Biology*, **jourvol** 3, **pages** 1–11, **february** 2007.

[17] S. M. Bohte, J. N. Kok **and** H. La Poutré, "Error-backpropagation in temporally encoded networks of spiking neurons," *Neurocomputing*, 2002.

[18] C. Eliasmith **and** C. H. Anderson, *Neural Engineering: Computation, Representation, and Dynamics in Neurobiological Systems*. MIT Press, 2003.

[19] G. Gallego **andothers**, "Event-based Vision: A Survey," *IEEE TPAMI*, 2020.

[20] E. O. Neftci, H. Mostafa **and** F. Zenke, "Surrogate Gradient Learning in Spiking Neural Networks," *CoRR*, **jourvol** abs/1901.09948, 2019.

[21] W. Fang **andothers**, "Incorporating Learnable Membrane Time Constant to Enhance Learning of Spiking Neural Networks," *Proceedings of ICCV*, 2021.

[22] G. Bellec **andothers**, "Long short-term memory and learning-to-learn in networks of spiking neurons," *NeurIPS*, 2018.

[23] E. M. Izhikevich, "Simple Model of Spiking Neurons," *IEEE Transactions on Neural Networks*, 2003.

[24] A. L. Hodgkin **and** A. F. Huxley, "A Quantitative Description of Membrane Current," *The Journal of Physiology*, 1952.

[25] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit **and**

N. Houlsby, *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*, 2021.

[26] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles **and** H. Jégou, *Training data-efficient image transformers & distillation through attention*, 2021.

[27] M. Pfeiffer **and** T. Pfeil, "Deep Learning With Spiking Neurons: Opportunities and Challenges," *Frontiers in Neuroscience*, **jourvol** Volume 12 - 2018, 2018.

[28] Z. Zhou, Y. Zhu, C. He, Y. Wang, S. Yan, Y. Tian **and** L. Yuan, *Spikformer: When Spiking Neural Network Meets Transformer*, 2022.

[29] W. Fang, Y. Chen, J. Ding, Z. Yu, T. Masquelier, D. Chen, L. Huang, H. Zhou, G. Li **and** Y. Tian, "SpikingJelly: An open-source machine learning infrastructure platform for spike-based intelligence," *Science Advances*, **jourvol** 9, **number** 40, eadi1480, 2023. eprint: `https://www.science.org/doi/pdf/10.1126/sciadv.adi1480`.

[30] T. Xiao, M. Singh, E. Mintun, T. Darrell, P. Dollár **and** R. Girshick, "Early convolutions help transformers see better," **in** *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, **volume** 34, 2021, **pages** 30 392–30 400.

[31] A. Hassani, S. Walton, N. Shah, A. Abuduweili, J. Li **and** H. Shi, "Escaping the big data paradigm with compact transformers," *arXiv preprint arXiv:2104.05704*, 2021.